

Model Binding and Validation

Theory

What is Model Binding?

- Model binding is the process ASP.NET Core uses to automatically convert HTTP request data (form fields, route values, query strings, headers) into strongly-typed .NET objects, so your controller action can work with C# objects instead of raw strings.

Why it matters: Without model binding, you'd have to manually parse `Request.Form["Name"]`, convert `Request.Form["Age"]` from string to int, handle missing/malformed data yourself, for every single field, on every single action. Model binding eliminates all of that boilerplate.

How it decides what to bind: The binder inspects your action method's parameter type. If it's a simple type (string, int, bool), it binds a single value. If it's a complex type (a class like `Student`), it inspects every **public property** on that class and tries to find a matching key in the incoming request data — matched **by name, case-insensitively**.

Binding sources, in order of precedence:

1. Route values (`{id}` in the URL pattern)
2. Form data (`Request.Form`) — this is what your registration form uses
3. Query string (`?name=value`)
4. Request body (for `[FromBody]`, typically JSON APIs)

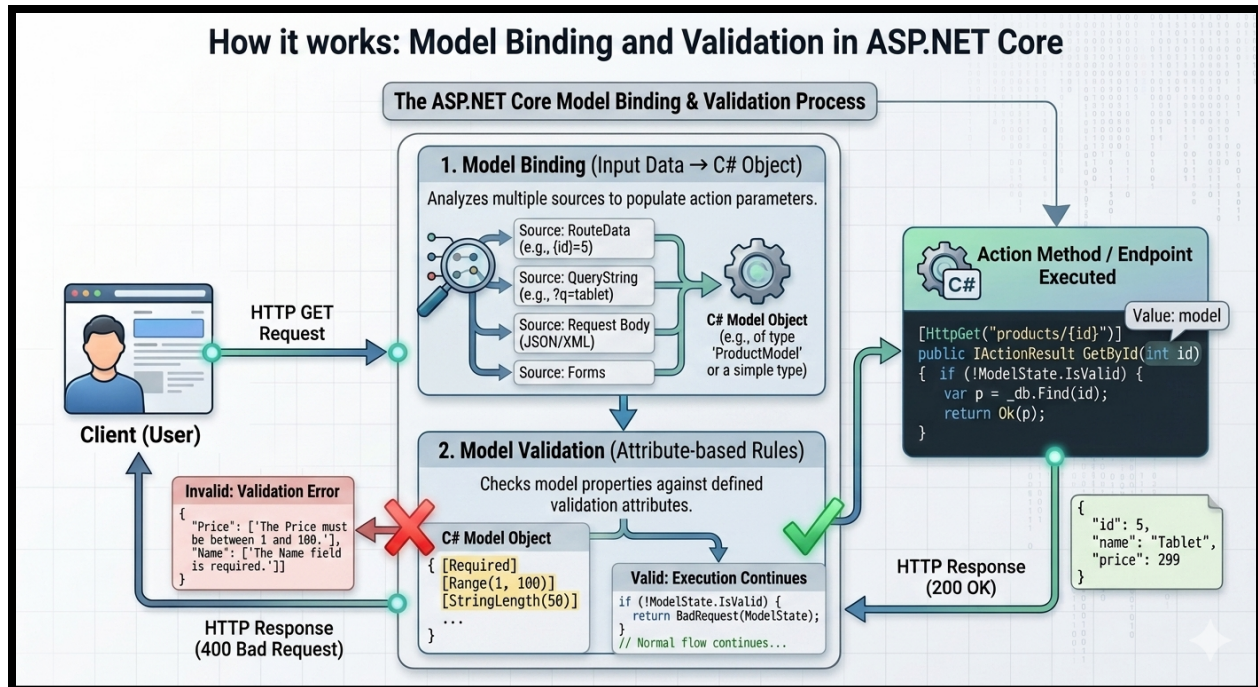
What is Model Validation?

Validation is the process of checking whether the bound data satisfies a set of business rules, *before* your action logic runs. In ASP.NET Core, this is typically done declaratively using **data annotation attributes** placed directly on model properties.

Why it matters: Validation logic lives in one place (the model) instead of being scattered across every controller action that happens to use that model. It's also automatically wired up to both server-side checks and client-side JavaScript validation, without writing that logic twice.

How it works internally: After binding populates your object, the framework runs a `Validator` over every property, checking each attribute's rule against the bound value. Every failure gets added to `ModelState`, a dictionary-like structure of `{ propertyName: [errorMessages] }`. Your action code then just asks `ModelState.IsValid` — a boolean that's true only if **every** property passed **every** rule.

How it works: Model Binding and Validation in ASP.NET Core



Property (binding)	Attributes (validation)
Id int	No validation needed
Name string	Required StringLength 2-50 chars
Age int	Required Range 5-100
Email string	Required EmailAddress format
Mobile string	Required Phone format

Syntax

1. Binding syntax — in the model

You don't write "binding code" in the model. You simply **name your properties to match your form fields**:

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
```

```

    public string Email { get; set; }
    public string Mobile { get; set; }
}

```

2. Binding syntax — in the controller

Binding is triggered by declaring a complex type as an action parameter:

```

[HttpPost]
public IActionResult Create(Student student)
{
    // 'student' arrives here already populated
}

```

You can also bind individual route/query values directly:

```

public IActionResult Details(int id)      // binds from route: /Student/Details/5
public IActionResult Search(string name)   // binds from query string: ?name=John

```

Explicit binding source attributes, when you need to override the default order:

```

csharp
public IActionResult Create([FromForm] Student student) // force form binding
public IActionResult Create([FromBody] Student student) // force JSON body binding (APIs)
public IActionResult Details([FromRoute] int id)
public IActionResult Search([FromQuery] string name)

```

3. Binding syntax — in the view

Tag helpers generate the exact name attribute the binder needs:

```
<input asp-for="Name" />
```

renders as:

```
<input type="text" id="Name" name="Name" value="" />
```

4. Validation syntax — data annotation attributes

All live in `System.ComponentModel.DataAnnotations`:

using `System.ComponentModel.DataAnnotations`;

```

public class Student
{
    [Required(ErrorMessage = "Name is required")]
    [StringLength(50, MinimumLength = 2)]
    public string Name { get; set; }

    [Required]
    [Range(5, 100, ErrorMessage = "Age must be between 5 and 100")]
    public int Age { get; set; }

    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    [Phone]

```

```
public string Mobile { get; set; }
}
```

Common built-in attributes:

Attribute	Purpose	Example
[Required]	Value must be provided	[Required]
[StringLength]	Max/min string length	[StringLength(50, MinimumLength = 2)]
[Range]	Numeric value must fall within bounds	[Range(5, 100)]
[EmailAddress]	Must match email pattern	[EmailAddress]
[Phone]	Must match phone pattern	[Phone]
[RegularExpression]	Custom regex match	[RegularExpression(@"^\d{10}\$")]
[Compare]	Must match another property	[Compare("Password")]

5. Validation syntax — in the controller

```
[HttpPost]
public IActionResult Create(Student student)
{
    if (ModelState.IsValid)
    {
        // all validation rules passed
        return RedirectToAction("Index");
    }

    // at least one rule failed — ModelState now contains the errors
    return View(student);
}

Reading specific errors manually (rarely needed, but good to know):
foreach (var error in ModelState["Name"].Errors)
{
    string message = error.ErrorMessage;
}
```

6. Validation syntax — in the view

```
<div asp-validation-summary="All"></div>
```

```
<input asp-for="Name" />
```

```
<span asp-validation-for="Name"></span>
```

Enabling client-side validation (in `_Layout.cshtml` or via `@section Scripts`):

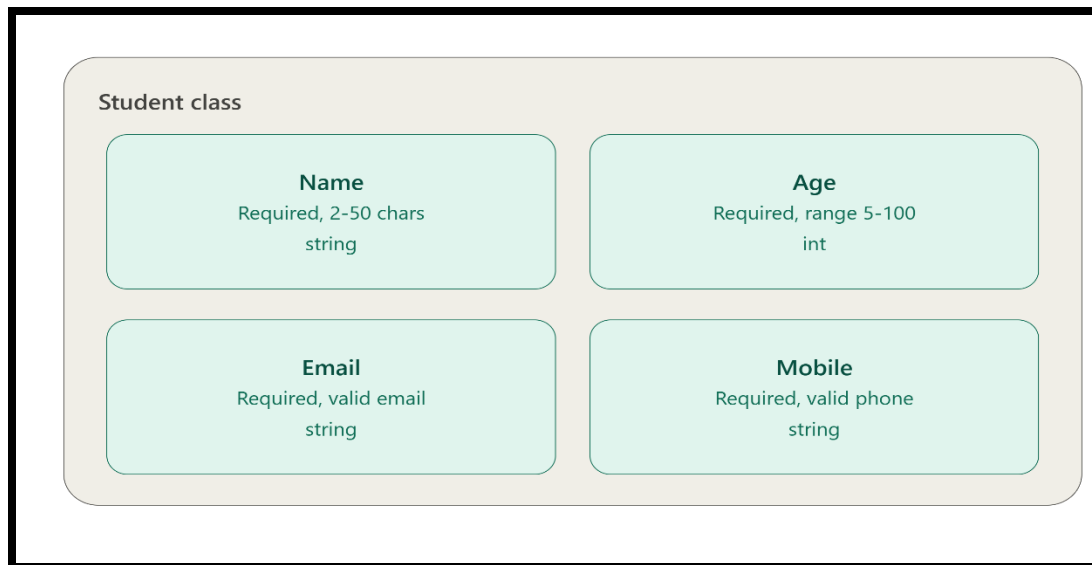
```
html
```

```
<script src="~/lib/jquery/dist/jquery.min.js"></script>
```

```
<script src="~/lib/jquery-validation/dist/jquery.validate.min.js"></script>
```

```
<script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.min.js"></script>
```

1. Models/Student.cs — the data shape and its rules



using System.ComponentModel.DataAnnotations;

namespace bindingandvalidation.Models

```
{
    public class Student
    {
        public int Id { get; set; }

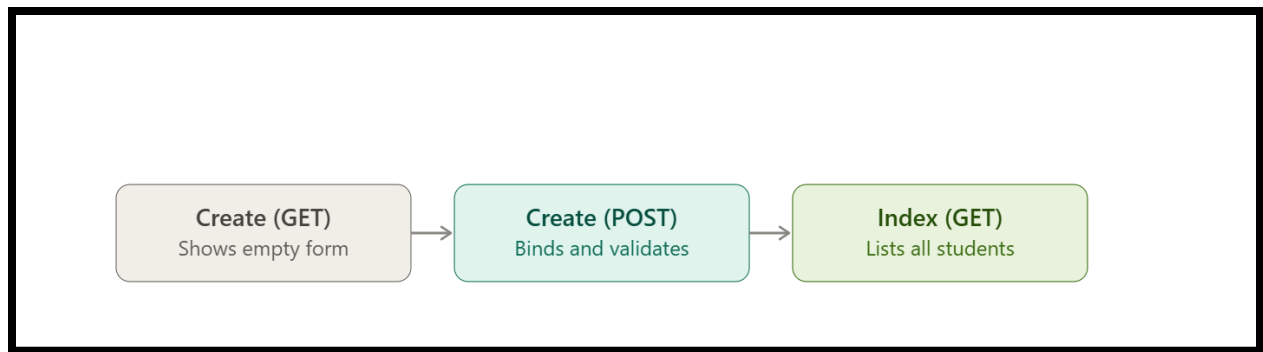
        [Required(ErrorMessage = "Name is required")]
        [StringLength(50, MinimumLength = 2, ErrorMessage = "Name must be between 2 and 50 characters")]
        [Display(Name = "Full Name")]
        public string Name { get; set; }

        [Required(ErrorMessage = "Age is required")]
        [Range(5, 100, ErrorMessage = "Age must be between 5 and 100")]
        public int Age { get; set; }

        [Required(ErrorMessage = "Email is required")]
        [EmailAddress(ErrorMessage = "Enter a valid email address")]
        public string Email { get; set; }

        [Required(ErrorMessage = "Mobile number is required")]
        [Phone(ErrorMessage = "Enter a valid mobile number")]
        [Display(Name = "Mobile Number")]
        public string Mobile { get; set; }
    }
}
```

2. Controllers/StudentController.cs — the three actions



```
using Microsoft.AspNetCore.Mvc;  
using bindingandvalidation.Models;  
using System.Collections.Generic;
```

```
namespace bindingandvalidation.Controllers  
{  
    public class StudentController : Controller  
    {  
        // Temporary in-memory storage (resets when the app restarts)  
        private static List<Student> _students = new List<Student>();  
  
        [HttpGet]  
        public IActionResult Create()  
        {  
            return View();  
        }  
  
        [HttpPost]  
        [ValidateAntiForgeryToken]  
        public IActionResult Create(Student student)  
        {  
            if (ModelState.IsValid)  
            {  
                student.Id = _students.Count + 1;  
                _students.Add(student);  
                return RedirectToAction("Index");  
            }  
  
            return View(student);  
        }  
  
        [HttpGet]  
        public IActionResult Index()  
        {
```

```

        return View(_students);
    }
}

```

3. Views/Student/Create.cshtml — the registration form

Here's a mockup of what the rendered form looks like:

The mockup shows a registration form with the following fields and values:

- Full name: Jane Doe
- Age: 22
- Email: jane@example.com (with a red error message: "Enter a valid email address")
- Mobile number: 9800000000

A "Register" button is located at the bottom of the form.

```
@model bindingandvalidation.Models.Student
```

```
@{
    ViewData["Title"] = "Student Registration";
}
```

```
<h2>Student Registration</h2>
```

```
<form asp-controller="Student" asp-action="Create" method="post">
    <div asp-validation-summary="All" style="color:red"></div>
```

```

    <div>
        <label asp-for="Name"></label>
        <input asp-for="Name" />
        <span asp-validation-for="Name" style="color:red"></span>
    </div>

```

```

<br />

<div>
    <label asp-for="Age"></label>
    <input asp-for="Age" />
    <span asp-validation-for="Age" style="color:red"></span>
</div>

<br />

<div>
    <label asp-for="Email"></label>
    <input asp-for="Email" />
    <span asp-validation-for="Email" style="color:red"></span>
</div>

<br />

<div>
    <label asp-for="Mobile"></label>
    <input asp-for="Mobile" />
    <span asp-validation-for="Mobile" style="color:red"></span>
</div>

<br />

<button type="submit">
    Register
</button>
</form>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}

```

4. Views/Student/Index.cshtml — the registered students list

Registered students

#	Name	Age	Email	Mobile
1	Subash Siwa	22	subash@example.com	9800000000
2	Jane Doe	24	jane@example.com	9811111111

```
@model List<bindingandvalidation.Models.Student>
```

```
@{  
    ViewData["Title"] = "Registered Students";  
}
```

```
<h2>Registered Students</h2>
```

```
<a asp-action="Create" class="btn btn-primary mb-3">Register New Student</a>
```

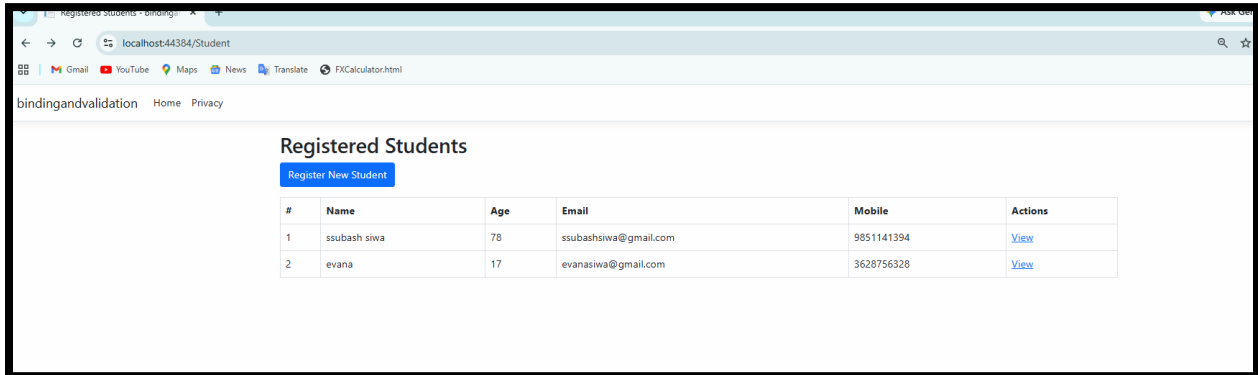
```
@if (Model.Count == 0)  
{  
    <p>No students registered yet.</p>  
}  
else  
{  
    <table class="table table-bordered">  
        <thead>  
            <tr>  
                <th>#</th>  
                <th>Name</th>  
                <th>Age</th>  
                <th>Email</th>  
                <th>Mobile</th>  
            </tr>  
        </thead>  
        <tbody>  
            @foreach (var s in Model)  
            {  
                <tr>  
                    <td>@s.Id</td>  
                    <td>@s.Name</td>  
                    <td>@s.Age</td>  
                    <td>@s.Email</td>
```

```

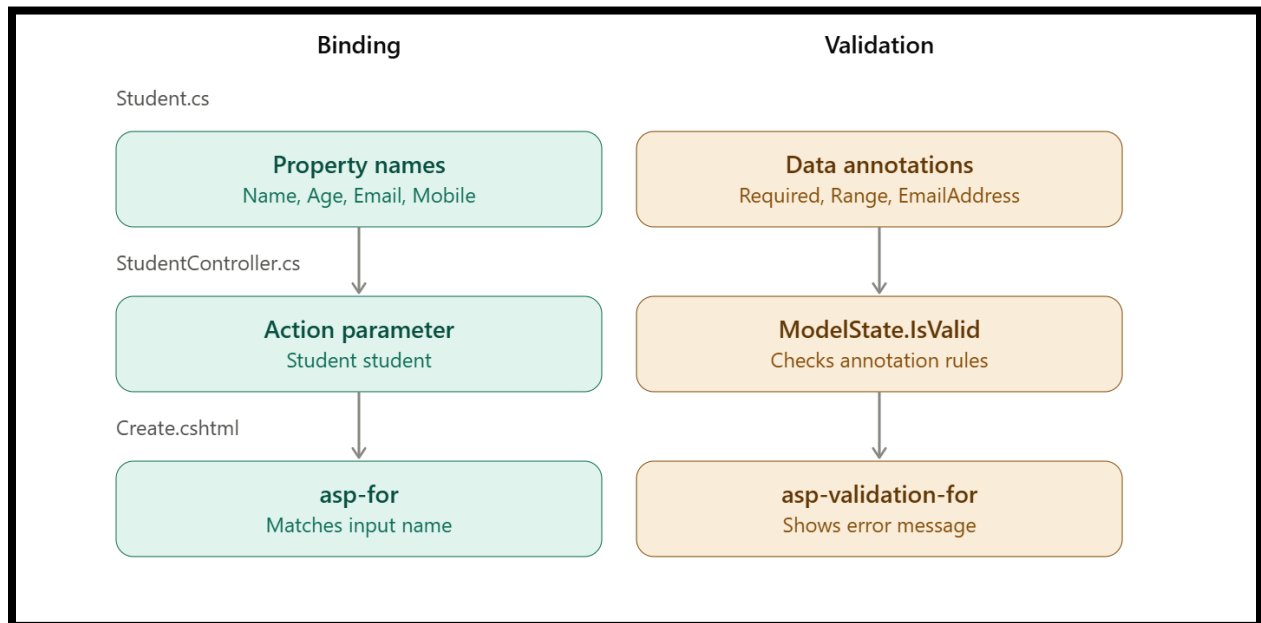
        <td>@s.Mobile</td>
    </tr>
}
</tbody>
</table>
}

```

That's the full set — model, controller, and both views, each paired with a visual of what it actually builds. Everything else (_ViewImports.cshtml, _Layout.cshtml script section, Program.cs) stays as given earlier in this conversation.



Here's the mapping — binding components in one column, validation components in the other, connected top to bottom showing the sequence each one runs in:



Reading it top to bottom, each column is a self-contained chain:

Binding chain (left): the property name in Student.cs is what the model binder matches against → the Student student parameter in the controller is what triggers that matching to happen → asp-for in the view is what generates the input's name attribute so it lines up correctly.

Validation chain (right): the [Required]/[Range]/etc. attributes in Student.cs define the rules → ModelState.IsValid in the controller is what actually runs those rules and checks the result → asp-validation-for in the view is what surfaces the resulting error message to the user.

Project on:

Question 1 — Student Registration with Conditional Validation

Scenario:

A college wants an online registration form for new students. The form collects the student's full name, age, email, mobile number, and course they wish to enroll in. The college has a rule: students under 18 years old must provide a parent/guardian email address, but students 18 or older don't need to provide one.

Task:

Create an ASP.NET Core MVC application that:

1. Defines a Student model with appropriate data annotations for Name, Age, Email, Mobile, and Course.
2. Enforces the conditional rule — if Age < 18, GuardianEmail must not be empty — using either IValidatableObject or a custom validation attribute.
3. Displays all validation errors on the form using tag helpers (asp-validation-for, asp-validation-summary).
4. On successful submission, redirects to a page listing all registered students.
5. Demonstrates what happens when: (a) all fields are valid, (b) a required field is missing, (c) a student under 18 doesn't provide a guardian email.

Question 2 — Product Order Form with Nested Model Binding

Scenario:

An online store wants a simple order form. Each order has a customer name, email, and a shipping address (street, city, postal code) as a nested object. The store requires the postal code to be exactly 5 digits, and the order quantity must be between 1 and 10 units.

Task:

Create an ASP.NET Core MVC application that:

1. Defines an Order model containing customer details and a nested Address model (Street, City, PostalCode).
2. Uses [RegularExpression] to enforce the 5-digit postal code format.
3. Uses [Range(1, 10)] to enforce the quantity limit.
4. Demonstrates nested object binding in the view using dot notation (asp-for="Address.City").
5. Shows the bound Order object (including its nested Address) in a confirmation view after successful validation, and shows appropriate error messages when validation fails (e.g. invalid postal code or out-of-range quantity).

complete guidelines for building either of the two scenario projects, structured as a step-by-step workflow you can follow from setup to submission.

1. Project setup

1. Open Visual Studio → **Create a new project** → **ASP.NET Core Web App (Model-View-Controller)**.

2. Name it something scenario-appropriate (e.g. StudentRegistrationSystem or OrderManagementSystem).
3. Choose **.NET 8.0** (or whatever LTS version your course requires) as the framework.
4. Leave **Authentication: None** unless your assignment specifically asks for login.

2. Plan the model before writing code

Before touching the editor, write out on paper (or in a comment block) exactly what fields you need and what rule applies to each one:

Field	Type	Rule
Name	string	Required, 2–50 chars
Age	int	Required, 5–100
Email	string	Required, valid email
GuardianEmail	string	Required only if Age < 18

Doing this mapping first prevents the common mistake of adding fields to the view before the model exists, which causes binding mismatches later.

3. Build the Models folder

1. Add a Models folder if it doesn't exist.
2. Create your main class (Student.cs or Order.cs) with data annotations matching your rule table.
3. If your scenario needs conditional or cross-field rules, implement `IValidatableObject` on the class.
4. If your scenario needs nested data (like an Address), create it as a separate small class and add it as a property on the main model.

4. Build the Controller

1. Add a Controller (Controllers folder → **Add** → **Controller** → **MVC Controller - Empty**).
2. Add a `[HttpGet]` action to show the empty form.
3. Add a `[HttpPost]` action with `[ValidateAntiForgeryToken]` that:
 - Checks `ModelState.IsValid`
 - On success: stores the object (in-memory list is fine for a class project) and redirects to a list/confirmation view
 - On failure: returns the same view with the invalid model, so errors display

5. Build the Views

1. Create a `Create.cshtml` matching your model — one `<div>` block per field, each with `asp-for`, a matching `<span asp-validation-for="...">`, and an `asp-validation-summary="All"` at the top.
2. Create an `Index.cshtml` (or `Confirmation.cshtml`) to display the saved records in a table.
3. `Confirm_ViewImports.cshtml` has `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers`.
4. `Confirm_Layout.cshtml` loads jQuery, jQuery Validation, and jQuery Validation Unobtrusive scripts so client-side validation fires instantly.

6. Test each rule deliberately

Don't just test with "correct" data. For each validation rule you defined, run one test that violates it, and confirm the exact error message appears:

- Leave a required field empty
- Enter an out-of-range number
- Enter a malformed email

- Trigger the conditional rule specifically (e.g. set Age to 15 with no guardian email)

Screenshot each of these — most instructors want proof the validation actually works, not just the happy path.

7. Document what you built

For submission, prepare a short write-up (this is often part of the grading rubric) covering:

- What the model represents and why each rule was chosen
- Which parts are binding vs validation (you can reuse the model/controller/view breakdown we discussed earlier)
- Screenshots of the success case and at least 2–3 failure cases
- Any advanced technique used (conditional validation, nested binding, custom regex) and why

8. Common pitfalls to avoid

- **Property name mismatch** — `asp-for="Name"` must match the model property exactly, case-sensitive in the generated name attribute matching.
- **Missing `@addTagHelper`** — silently breaks every `asp-*` attribute with no error message.
- **Forgetting `[ValidateAntiForgeryToken]` mismatch** — if the form doesn't include the anti-forgery token (it does automatically via `<form asp-action="...">`), POST requests will be rejected with a 400 error.
- **Testing only the happy path** — a project that only ever submits valid data doesn't actually demonstrate validation is working.

What to include in your Output section

1. Screenshot: Empty form (initial load)

Shows the registration/order form freshly loaded, before any data entry. Caption it something like: *"Figure 1: Initial registration form"*.

2. Screenshot: Successful submission (happy path)

Fill in all fields correctly, submit, and capture the resulting confirmation/list page showing the new record. Caption: *"Figure 2: Successful registration — redirected to student list"*.

3. Screenshot: Required field validation error

Leave one required field blank (e.g. Name), submit, and capture the red error message appearing under that field plus in the validation summary. Caption: *"Figure 3: Required field validation triggered"*.

4. Screenshot: Format/range validation error

Enter an invalid format (bad email, out-of-range age, wrong postal code pattern) and capture the specific error message shown. Caption: *"Figure 4: Format validation triggered (invalid email)"*.

5. Screenshot: Conditional/advanced rule (if you implemented one)

If you added `IsValidatableObject` logic (e.g. guardian email required under 18), show that specific scenario triggering its custom error message. Caption: *"Figure 5: Conditional validation — guardian email required for age under 18"*.

6. Screenshot: Final data table/list view

Show the Index view with 2–3 successfully registered records, proving data persists correctly across multiple submissions in the same session. Caption: *"Figure 6: Registered students list"*.

Suggested Output section structure

Output

1. Initial form

[screenshot]

The form loads with empty fields and no validation messages shown.

2. Successful registration

[screenshot]

Submitting valid data redirects the user to the student list, confirming the record was saved with correctly bound values.

3. Validation — missing required field

[screenshot]

Leaving "Name" empty triggers "Name is required" both inline and in the validation summary.

4. Validation — invalid format

[screenshot]

Entering "notanemail" in the Email field triggers "Enter a valid email address."

5. Conditional validation

[screenshot]

A student aged 15 without a guardian email triggers the custom rule: "Guardian email is required for students under 18."

6. Final registered list

[screenshot]

All successfully validated students appear in the table with correct values.

URL Routing in ASP.NET Core

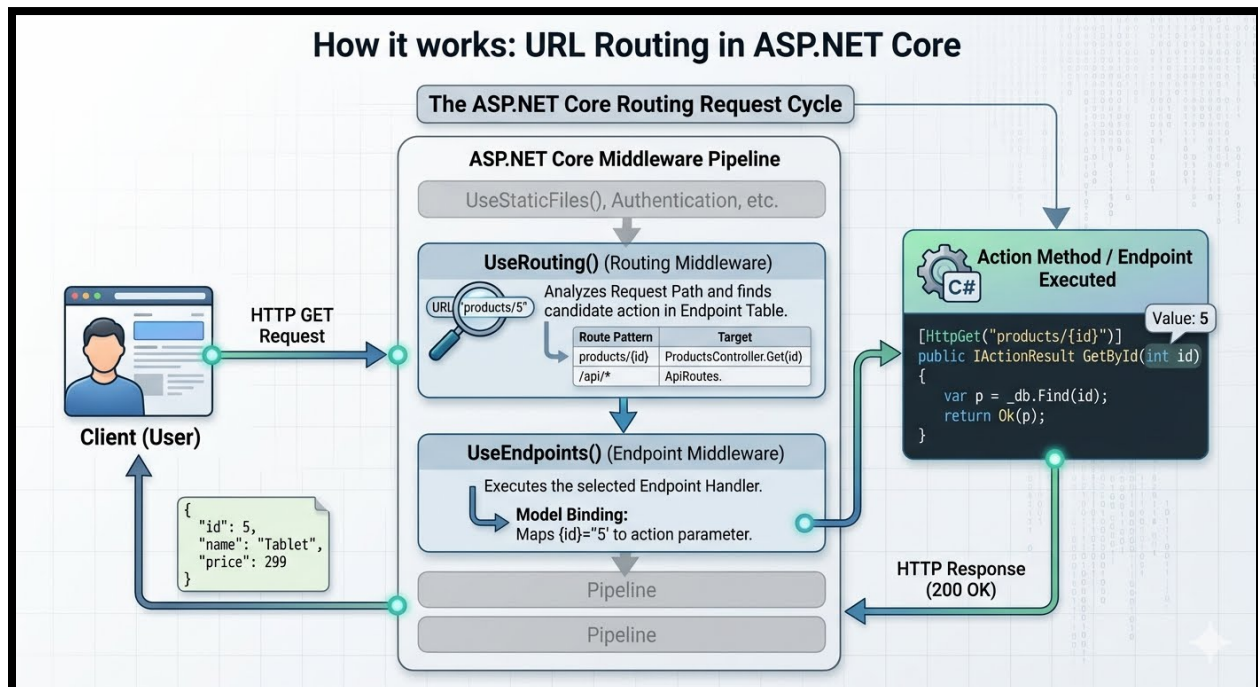
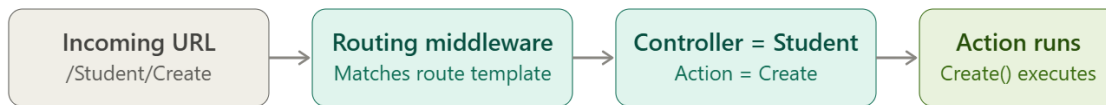
Theory

URL routing is the mechanism that maps an incoming HTTP request URL to a specific controller action method. Without routing, the framework wouldn't know that a request to /Student/Create should execute StudentController.Create() — routing is the translation layer between "what the browser asked for" and "what code should run."

ASP.NET Core supports two routing approaches:

1. **Conventional routing** — one general pattern defined once, applied to all controllers (what your project currently uses)
2. **Attribute routing** — routes defined individually on each action method, giving fine-grained control

Here's how a request travels through the routing system:



Theory

URL routing maps an incoming request URL to a specific controller action. Two approaches exist:

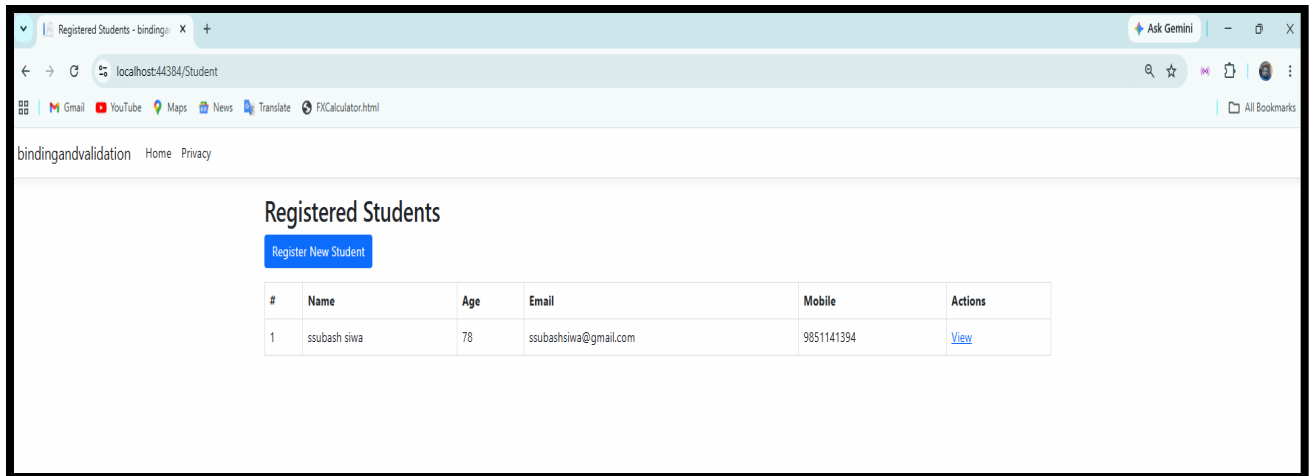
- **Conventional routing** — one pattern defined once in Program.cs, applied to every controller (this is what your project currently uses).
- **Attribute routing** — routes defined individually on each action, giving fine-grained control.

1.Syntax — Conventional routing (your current setup)

In Program.cs:

```

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
  
```



This single line handles every URL in your app by pattern-matching three URL segments:

URL	controller	action	id
/	Home (default)	Index (default)	—
/Student/Create	Student	Create	—
/Student/Details/ 5	Student	Details	5

The ? after id marks it **optional** — routes without a third segment still match.

Student/Details/5 would currently 404. Here's how to add it so that URL works and shows one specific student.

Modify code previous code:

Step 1 — Add the Details action to StudentController.cs

[HttpGet]

public IActionResult Details(int id)

{

var student = _students.FirstOrDefault(s => s.Id == id);

if (student == null)

{

return NotFound(); // returns HTTP 404 if no student with that id exists

}

return View(student);

}

Make sure using **System.Linq;**

is at the top of the file (needed for FirstOrDefault).

Your full controller now looks like:

using Microsoft.AspNetCore.Mvc;

using bindingandvalidation.Models;

using System.Collections.Generic;

```

using System.Linq;

namespace bindingandvalidation.Controllers
{
    public class StudentController : Controller
    {
        private static List<Student> _students = new List<Student>();

        [HttpGet]
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public IActionResult Create(Student student)
        {
            if (ModelState.IsValid)
            {
                student.Id = _students.Count + 1;
                _students.Add(student);
                return RedirectToAction("Index");
            }

            return View(student);
        }

        [HttpGet]
        public IActionResult Index()
        {
            return View(_students);
        }

        [HttpGet]
        public IActionResult Details(int id)
        {
            var student = _students.FirstOrDefault(s => s.Id == id);

            if (student == null)
            {
                return NotFound();
            }
        }
    }
}

```

```

        return View(student);
    }
}

```

Step 2 — Create the view Views/Student/Details.cshtml

@model bindingandvalidation.Models.Student

```

@{
    ViewData["Title"] = "Student Details";
}

```

<h2>Student Details</h2>

```

<dl class="row">
    <dt class="col-sm-2">Id</dt>
    <dd class="col-sm-10">@Model.Id</dd>

    <dt class="col-sm-2">Name</dt>
    <dd class="col-sm-10">@Model.Name</dd>

    <dt class="col-sm-2">Age</dt>
    <dd class="col-sm-10">@Model.Age</dd>

    <dt class="col-sm-2">Email</dt>
    <dd class="col-sm-10">@Model.Email</dd>

    <dt class="col-sm-2">Mobile</dt>
    <dd class="col-sm-10">@Model.Mobile</dd>
</dl>

```

<a asp-action="Index">Back to list

Step 3 — Link to it from Index.cshtml

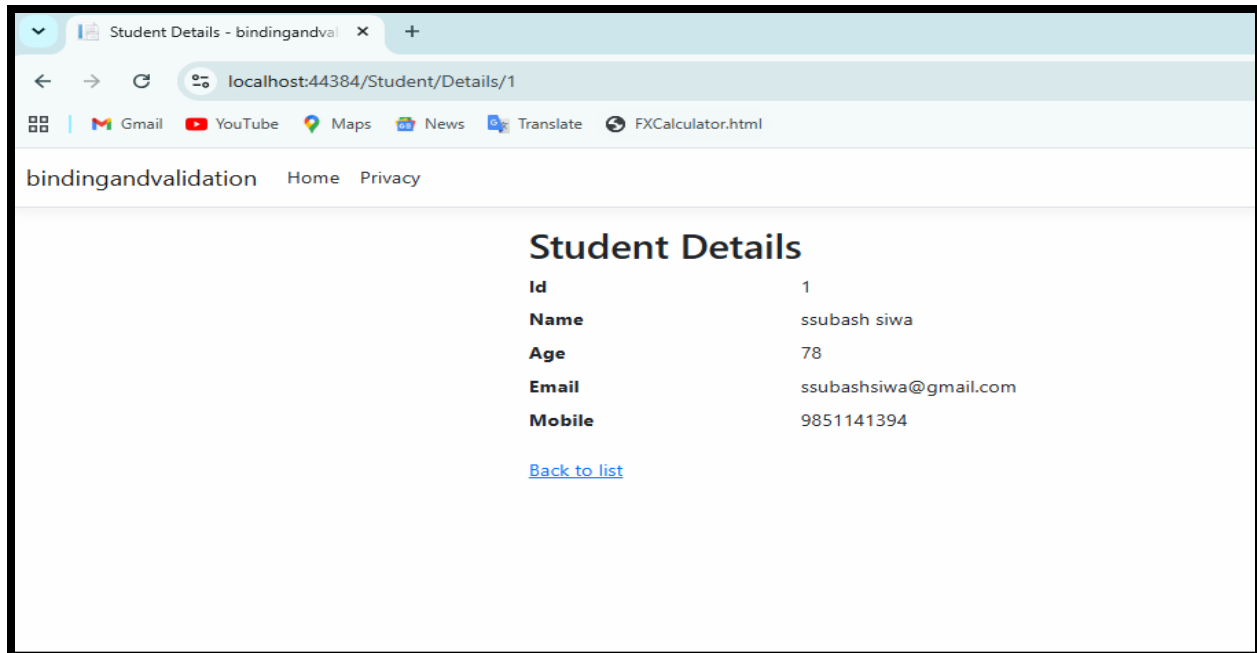
So you don't have to type the URL manually, add a "View" link per row:

```

<td>
    <a asp-action="Details" asp-route-id="@s.Id">View</a>
</td>

```

asp-route-id is the tag helper syntax for supplying a route parameter — it generates href="/Student/Details/3" for a student with Id=3.



Syntax — Attribute routing (alternative/complementary approach)

Applied directly above the controller class or action method:

```
[Route("students")]
```

```
public class StudentController : Controller
```

```
{
```

```
    [Route("register")]           // matches /students/register
```

```
    public IActionResult Create() { ... }
```

```
    [Route("details/{id}")]       // matches /students/details/5
```

```
    public IActionResult Details(int id) { ... }
```

```
    [Route("api/students/{id:int}")] // route constraint: id must be an int
```

```
    public IActionResult ApiDetails(int id) { ... }
```

```
}
```

2. Attribute Routing

Theory

Attribute routing lets you define routes directly on controllers and action methods using `[Route]` attributes, instead of relying on one general pattern in `Program.cs`. This gives you precise control over the exact URL shape for each action — useful when you want URLs that don't follow the default `{controller}/{action}/{id}` convention (e.g. REST-style APIs, SEO-friendly URLs, versioned endpoints).

Key difference from conventional routing:

	Conventional routing	Attribute routing
Defined	Once, in <code>Program.cs</code>	On each controller/action
Applies to	All controllers automatically	Only controllers/actions you annotate
Best for	Standard CRUD-style MVC	APIs, custom URL structures, fine

	apps	control
Flexibility	One-size-fits-all pattern	Per-action customization

Both can coexist in the same project — this is exactly your setup, since MVC apps typically use conventional routing for views and attribute routing for API controllers.

Syntax

1. Basic [Route] on an action

```
public class StudentController : Controller
{
    [Route("student/register")]
    public IActionResult Create()
    {
        return View();
    }
}
```

This makes /student/register load your Create action — **not** /Student/Create. The conventional route's {controller}/{action} pattern is bypassed entirely for this action.

2. [Route] on the controller — sets a shared prefix

```
[Route("students")]
public class StudentController : Controller
{
    [Route("register")] // → /students/register
    public IActionResult Create() { ... }

    [Route("list")] // → /students/list
    public IActionResult Index() { ... }

    [Route("{id}")] // → /students/5
    public IActionResult Details(int id) { ... }
}
```

3. Route tokens — avoid hardcoding names

```
[Route("[controller]/[action]")]
public class StudentController : Controller
{
    public IActionResult Create() { ... } // → /Student/Create
    public IActionResult Index() { ... } // → /Student/Index
}
```

[controller] and [action] are placeholders the framework fills in automatically from the class/method name — so if you rename the controller later, the route updates itself.

4. HTTP verb attributes combined with routes

```
[HttpGet("students/create")]
public IActionResult Create() { ... }

[HttpPost("students/create")]
```

```
[ValidateAntiForgeryToken]
```

```
public IActionResult Create(Student student) { ... }
```

[HttpGet("...")] and [HttpPost("...")] combine the HTTP verb restriction and the route path into a single attribute — this is the most common pattern in real projects, especially Web APIs.

5. Route constraints (same syntax as conventional routing)

```
[HttpGet("students/{id:int}")]
```

```
public IActionResult Details(int id) { ... }
```

```
[HttpGet("students/{id:int:min(1)}")]
```

```
public IActionResult DetailsPositive(int id) { ... }
```

6. Multiple routes to the same action

```
[HttpGet("students/{id}")]
```

```
[HttpGet("students/details/{id}")]
```

```
public IActionResult Details(int id) { ... }
```

Both URLs now resolve to the same action — useful for supporting legacy URLs alongside a new format.

Applying it to your StudentController

If you wanted cleaner, REST-style URLs instead of the default /Student/Create shape:

```
using Microsoft.AspNetCore.Mvc;
```

```
using bindingandvalidation.Models;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
namespace bindingandvalidation.Controllers
```

```
{
```

```
    [Route("students")]
```

```
    public class StudentController : Controller
```

```
    {
```

```
        private static List<Student> _students = new List<Student>();
```

```
        [HttpGet("register")]
```

```
        public IActionResult Create()
```

```
        {
```

```
            return View();
```

```
        }
```

```
        [HttpPost("register")]
```

```
        [ValidateAntiForgeryToken]
```

```
        public IActionResult Create(Student student)
```

```
        {
```

```
            if (ModelState.IsValid)
```

```
            {
```

```
                student.Id = _students.Count + 1;
```

```
                _students.Add(student);
```

```
                return RedirectToAction("Index");
```

```

    }
    return View(student);
}

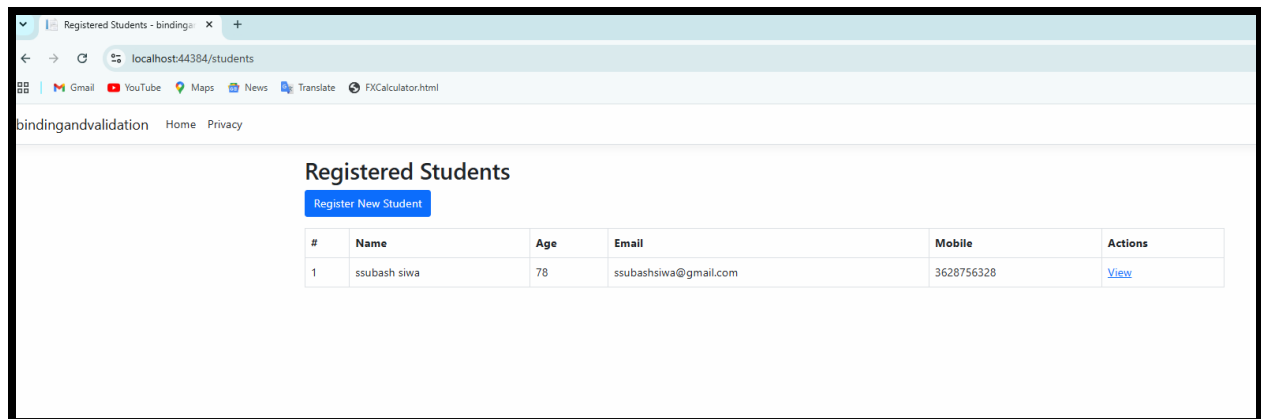
[HttpGet("")]
public IActionResult Index()
{
    return View(_students);
}

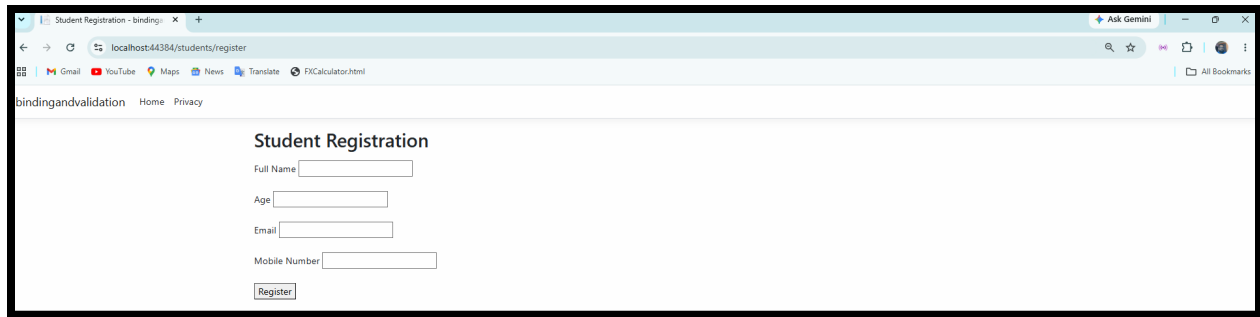
[HttpGet("{id:int}")]
public IActionResult Details(int id)
{
    var student = _students.FirstOrDefault(s => s.Id == id);
    if (student == null) return NotFound();
    return View(student);
}
}
}

```

This changes your URLs to:

Old (conventional)	New (attribute)
/Student/Create	/students/register
/Student/Index	/students
/Student/Details/5	/students/5





Key routing features

1. Route parameters — placeholders in { } captured from the URL and passed as method arguments:

csharp

```
[Route("details/{id}")]
```

```
public IActionResult Details(int id)
```

2. Route constraints — restrict what a parameter can match:

csharp

```
{id:int} // must be an integer
```

```
{id:alpha} // must be alphabetic
```

```
{id:minlength(3)} // minimum length
```

```
{id:regex(^d{4}$)} // custom pattern
```

3. Default values — {controller=Home}/{action=Index} means if a segment is missing, it falls back to the default.

4. Optional parameters — {id?} means the segment can be omitted entirely.

5. Route order/priority — attribute routes are matched before falling back to conventional routing; more specific routes should be declared first when ambiguity is possible.

6. Generating URLs from routes (instead of hardcoding them) — tag helpers use the route system in reverse:

```
<a asp-controller="Student" asp-action="Create">Register</a>
```

This generates /Student/Create by asking the routing system "what URL matches this controller+action?"

— so if you ever change your route pattern, all your links update automatically without hardcoding href="/Student/Create" anywhere.

Applied to your project: when someone requests /Student/Create, the router splits it into controller=Student, action=Create, no id — then instantiates StudentController and calls Create(). When they POST back to the same URL, the same routing rule applies, but ASP.NET Core distinguishes the GET and POST versions via the [HttpGet]/[HttpPost] attributes on the actions themselves, not via the route pattern.

Project on:

Scenario: Online Store Order Form

An online store needs an order form where customers enter:

- Customer Name
- Email
- Product Quantity
- Shipping Address:

- Street
- City
- Postal Code

The system validates:

- Postal Code must contain exactly **5 digits**
- Quantity must be between **1 and 10**

After successful validation, the entered order details are displayed in a confirmation page.

Web API Applications: API Controllers

Theory

An API Controller is a controller designed to return data (typically JSON or XML) instead of HTML views. It's the backbone of a Web API project — used when your app serves mobile apps, JavaScript frontends (React, Angular), or other services, rather than rendering server-side Razor views.

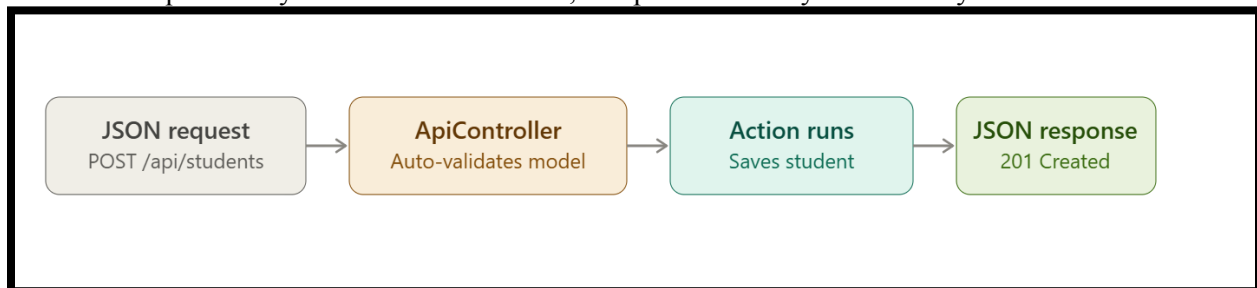
Key differences from an MVC controller:

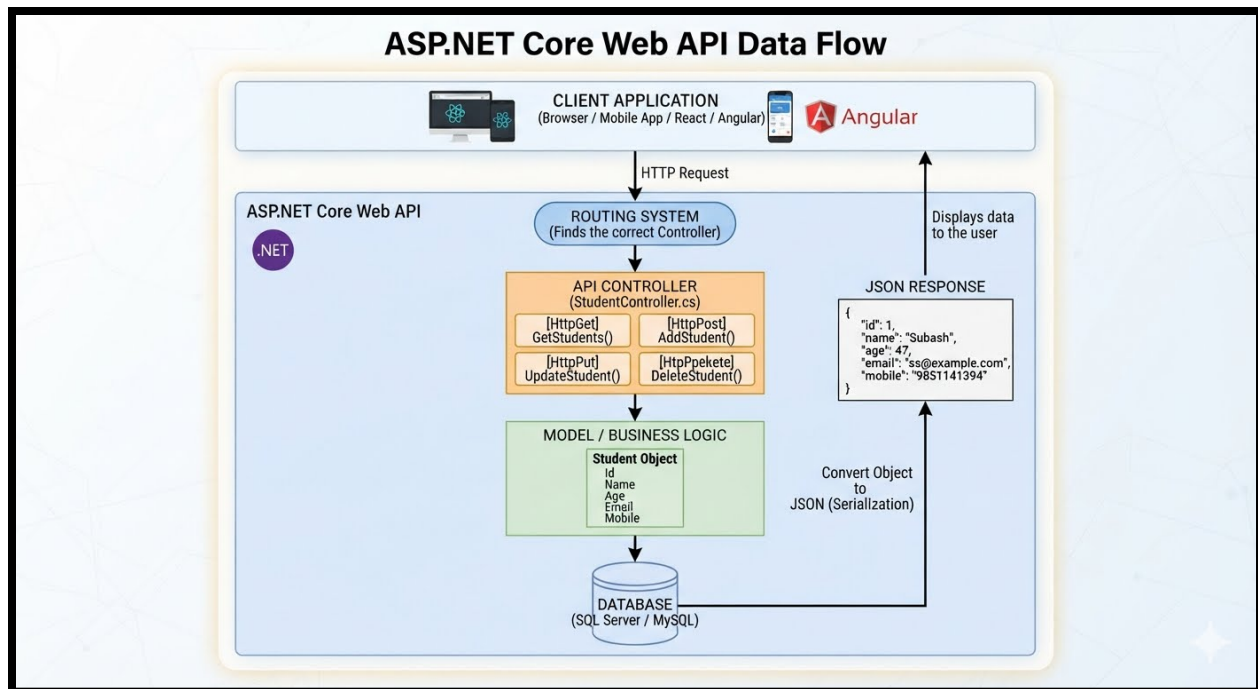
	MVC Controller	API Controller
Base class	Controller	ControllerBase (lighter — no view-rendering support)
Returns	IActionResult → Views (.cshtml)	IActionResult / ActionResult<T> → JSON/XML
Attribute	none required	[ApiController] (enables extra API behaviors)
Routing	Usually conventional	Usually attribute routing ([Route])
Model binding source	Form data by default	JSON body by default ([FromBody])

The [ApiController] attribute is what unlocks several automatic behaviors:

- **Automatic model validation** — if ModelState.IsValid is false, it automatically returns a 400 Bad Request with error details, without you writing the if check yourself.
- **Automatic binding source inference** — complex types are assumed to come from the request body; simple types from the route/query, without needing explicit [FromBody]/[FromQuery] in most cases.
- **Requires attribute routing** — conventional routing isn't supported on [ApiController]-decorated classes; every controller needs a [Route].

Here's the request lifecycle in an API controller, compared to what you've already built for MVC:





Syntax

1. Minimal API controller structure

using Microsoft.AspNetCore.Mvc;

```
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    // actions go here
}
```

- `[ApiController]` — enables the automatic behaviors described above.
- `[Route("api/[controller]")]` — sets the base path; `[controller]` resolves to `Students` (from `StudentsController`), giving you `/api/students`.
- `ControllerBase` — lighter base class without view support, since API controllers never return HTML.

2. GET endpoints

```
[HttpGet]
public ActionResult<List<Student>> GetAll()
{
    return Ok(_students);
}
```

```
[HttpGet("{id:int}")]
public ActionResult<Student> GetById(int id)
{
}
```

```

    var student = _students.FirstOrDefault(s => s.Id == id);
    if (student == null) return NotFound();
    return Ok(student);
}

```

ActionResult<T> lets you return either the actual data (Ok(student)) or an HTTP status result (NotFound()) from the same method — the framework figures out which one you meant.

3. POST endpoint (create)

```

[HttpPost]
public ActionResult<Student> Create(Student student)
{
    // No need to check ModelState.IsValid — [ApiController] does it automatically
    student.Id = _students.Count + 1;
    _students.Add(student);

    return CreatedAtAction(nameof(GetById), new { id = student.Id }, student);
}

```

CreatedAtAction returns a 201 Created response, with a Location header pointing to the new resource's URL (built from the GetById route) and the created object in the body.

4. PUT endpoint (update)

```

[HttpPut("{id:int}")]
public IActionResult Update(int id, Student updatedStudent)
{
    var student = _students.FirstOrDefault(s => s.Id == id);
    if (student == null) return NotFound();

    student.Name = updatedStudent.Name;
    student.Age = updatedStudent.Age;
    student.Email = updatedStudent.Email;
    student.Mobile = updatedStudent.Mobile;

    return NoContent(); // 204 - success, nothing to return
}

```

5. DELETE endpoint

```

[HttpDelete("{id:int}")]
public IActionResult Delete(int id)
{
    var student = _students.FirstOrDefault(s => s.Id == id);
    if (student == null) return NotFound();

    _students.Remove(student);
    return NoContent();
}

```

6. Explicit binding source attributes

```

public IActionResult Search(

```

[FromQuery] string name, // ?name=John
 [FromRoute] int id, // /students/5
 [FromBody] Student student, // JSON in request body
 [FromHeader] string apiKey) // custom header value

7. Common IActionResult / ActionResult<T> helper methods

Method	HTTP status	Use case
Ok(data)	200	Successful GET
CreatedAtAction(...)	201	Successful POST
NoContent()	204	Successful PUT/DELETE with no body
BadRequest()	400	Manual validation failure
NotFound()	404	Resource doesn't exist
Unauthorized()	401	Missing/invalid auth
Forbidden()	403	Authenticated but not permitted
Conflict()	409	Duplicate/conflicting resource
StatusCode(500)	custom	Any other status code

Full example — converting your Student project to a Web API controller

```
using Microsoft.AspNetCore.Mvc;
using bindingandvalidation.Models;
using System.Collections.Generic;
using System.Linq;
```

```
namespace bindingandvalidation.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class StudentsController : ControllerBase
    {
        private static List<Student> _students = new List<Student>();

        [HttpGet]
        public ActionResult<List<Student>> GetAll()
        {
            return Ok(_students);
        }

        [HttpGet("{id:int}")]
        public ActionResult<Student> GetById(int id)
        {
            var student = _students.FirstOrDefault(s => s.Id == id);
            if (student == null) return NotFound();
            return Ok(student);
        }
    }
}
```

```

    }

    [HttpPost]
    public ActionResult<Student> Create(Student student)
    {
        student.Id = _students.Count + 1;
        _students.Add(student);
        return CreatedAtAction(nameof(GetById), new { id = student.Id }, student);
    }

    [HttpPut("{id:int}")]
    public IActionResult Update(int id, Student updated)
    {
        var student = _students.FirstOrDefault(s => s.Id == id);
        if (student == null) return NotFound();

        student.Name = updated.Name;
        student.Age = updated.Age;
        student.Email = updated.Email;
        student.Mobile = updated.Mobile;
        return NoContent();
    }

    [HttpDelete("{id:int}")]
    public IActionResult Delete(int id)
    {
        var student = _students.FirstOrDefault(s => s.Id == id);
        if (student == null) return NotFound();

        _students.Remove(student);
        return NoContent();
    }
}

```

Testing this requires a tool like **Postman** or **curl** (or Swagger UI, if enabled in Program.cs with `builder.Services.AddSwaggerGen()`), since there's no HTML form — you'd POST raw JSON like `{"name": "Jane", "age": 22, "email": "jane@x.com", "mobile": "9800000000"}` directly to `/api/students`.

Architected Swagger integration strategy for MVC project setup

Architected Swagger integration strategy for MVC project setup

Enabling Swagger / OpenAPI

Swagger (via the **Swashbuckle.AspNetCore** package) auto-generates an interactive web UI from your API controllers, letting you test GET/POST/PUT/DELETE endpoints directly in the browser — no Postman needed.

Step 1 — Install the NuGet package

In Visual Studio: **Tools** → **NuGet Package Manager** → **Manage NuGet Packages for Solution** → search for Swashbuckle.AspNetCore → Install.

Or via Package Manager Console:

Install-Package Swashbuckle.AspNetCore

Or via .NET CLI:

dotnet add package Swashbuckle.AspNetCore

Step 2 — Register Swagger services in Program.cs

Add these two lines **before** builder.Build():

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.AddControllersWithViews();
```

// Add these two lines

```
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen();
```

```
var app = builder.Build();
```

Step 3 — Enable the Swagger middleware

Add this **after** app.Build(), typically right after the environment check:

```
if (!app.Environment.IsDevelopment())
```

```
{
```

```
    app.UseExceptionHandler("/Home/Error");
```

```
    app.UseHsts();
```

```
}
```

// Add these two lines

```
if (app.Environment.IsDevelopment())
```

```
{
```

```
    app.UseSwagger();
```

```
    app.UseSwaggerUI();
```

```
}
```

```
app.UseHttpsRedirection();
```

```
app.UseStaticFiles();
```

```
app.UseRouting();
```

```
app.UseAuthorization();
```

```
app.MapControllerRoute(
```

```
    name: "default",
```

```
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

```
app.Run();
```

Wrapping it in IsDevelopment() is standard practice — you typically don't want Swagger exposed in production.

Step 4 — Make sure your API controller has both routing attributes

This is the StudentsController from before — Swagger only picks up controllers with [ApiController] + [Route]:

```
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    // GetAll, GetById, Create, Update, Delete actions
}
```

Step 5 — Run and open Swagger UI

1. **Ctrl+F5** to run the app.
2. Navigate to:
<https://localhost:XXXXXX/swagger>
3. You'll see a page listing every endpoint on StudentsController — GET /api/students, GET /api/students/{id}, POST /api/students, PUT /api/students/{id}, DELETE /api/students/{id} — each expandable with a **"Try it out"** button.

Step 6 — Test an endpoint from the UI

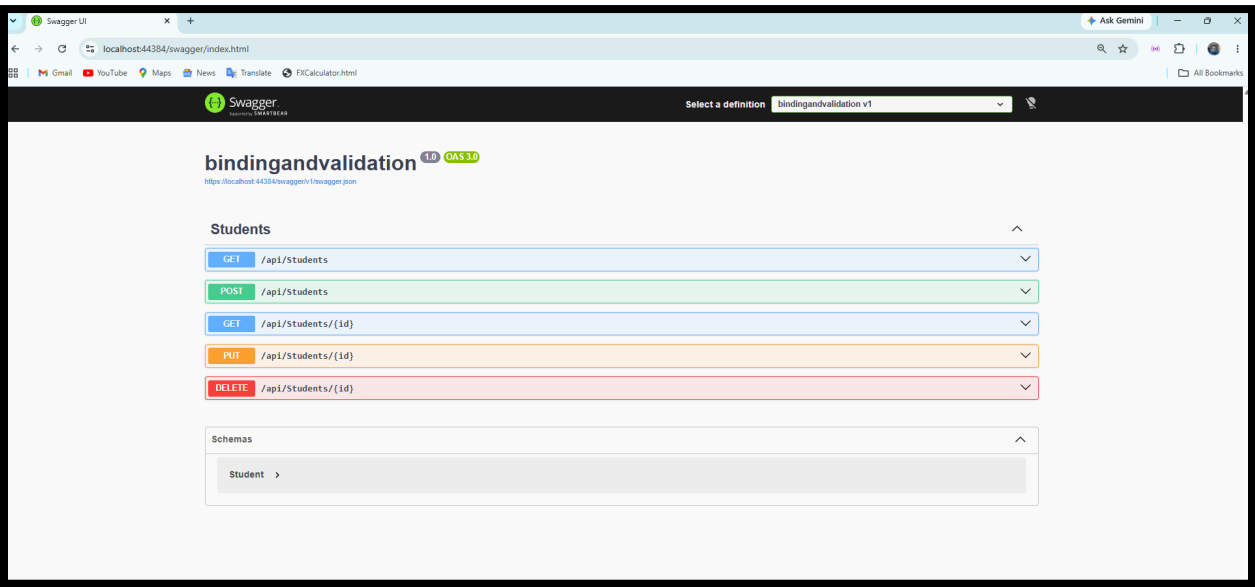
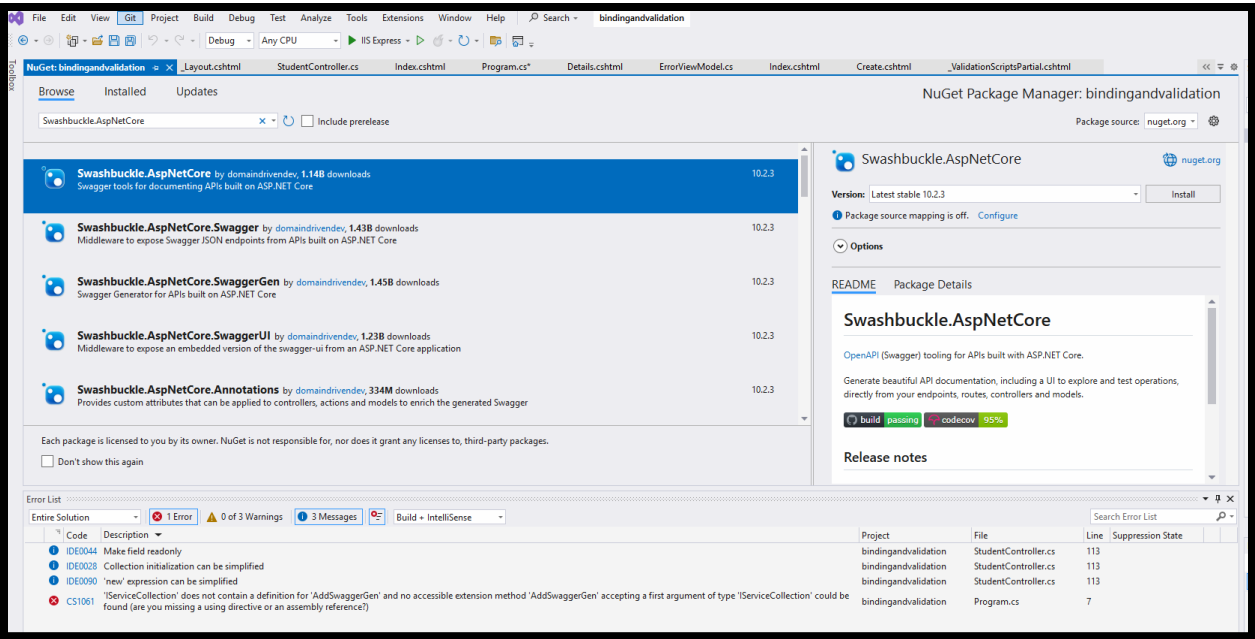
1. Click **POST /api/students** → **Try it out**.
2. Swagger shows a JSON template you can edit:

```
json
{
  "id": 0,
  "name": "string",
  "age": 0,
  "email": "string",
  "mobile": "string"
}
```

3. Replace with real values:

```
json
{
  "id": 0,
  "name": "Jane Doe",
  "age": 22,
  "email": "jane@example.com",
  "mobile": "9800000000"
}
```

4. Click **Execute** — you'll see the actual HTTP response (status code, headers, and the returned JSON body) right below.



	response_1784451546884.json	7/19/2026 2:44 PM	JSON Source File	1 KB
	response_1784452499645.json	7/19/2026 3:00 PM	JSON Source File	1 KB
	response_1784451194203.json	7/19/2026 2:41 PM	JSON Source File	1 KB

```
1 {
2   "id": 2,
3   "name": "subash",
4   "age": 47,
5   "email": "ss@example.com",
6   "mobile": "9851141394"
7 }
```

Students

GET /api/Students

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:44384/api/Students' \
  -H 'accept: text/plain'
```

Request URL

```
https://localhost:44384/api/Students
```

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "name": "subash",
  "age": 47,
  "email": "ss@example.com",
  "mobile": "9851141394"
},
{
  "id": 2,
  "name": "subash",
  "age": 47,
  "email": "ss@example.com",
  "mobile": "9851141394"
}
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 19 Jul 2026 09:29:46 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

POST/api/Students

Parameters

CancelReset

No parameters

Request body

application/json

Edit Value | Schema

```
{
  "id": 3,
  "name": "sushant",
  "age": 24,
  "email": "sushantsiwa@gmail.com",
  "mobile": "982333456"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:44384/api/Students' \
  -H 'accept: text/plain' \
  -H 'content-type: application/json' \
  -d '{
    "id": 3,
    "name": "sushant",
    "age": 24,
    "email": "sushantsiwa@gmail.com",
    "mobile": "982333456"
  }'
```

Curl

```
curl -X 'POST' \
  'https://localhost:44384/api/Students' \
  -H 'accept: text/plain' \
  -H 'content-type: application/json' \
  -d '{
    "id": 3,
    "name": "sushant",
    "age": 24,
    "email": "sushantsiwa@gmail.com",
    "mobile": "982333456"
  }'
```

Request URL

https://localhost:44384/api/Students

Server response

CodeDetails

201

Undocumented

Response body

```
{
  "id": 3,
  "name": "sushant",
  "age": 24,
  "email": "sushantsiwa@gmail.com",
  "mobile": "982333456"
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 19 Jul 2026 09:32:26 GMT
location: https://localhost:44384/api/Students/3
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code	Description	Links
200	OK	No links

Media type

text/plain

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "name": "string",
  "age": 100,
  "email": "user@example.com",
  "mobile": "string"
}
```

GET /api/Students/{id}

Parameters

Name

Description

id * required

integer(\$int32)

(path)

2

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:44384/api/Students/2' \
  -H 'accept: text/plain'
```

Request URL

https://localhost:44384/api/Students/2

Server response

Code

Details

200

Response body

```
{
  "id": 2,
  "name": "Subash",
  "age": 47,
  "email": "ss@example.com",
  "mobile": "9851141394"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 19 Jul 2026 09:34:03 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code

Description

Links

Responses

Code

Description

Links

200

OK

No links

Media type

text/plain

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "name": "string",
  "age": 100,
  "email": "user@example.com",
  "mobile": "string"
}
```

Full description:

The purpose of this page is to **test, document, and understand your API without using a separate tool like Postman.**

Your API is a **Student CRUD API.**

1. GET /api/Students

Purpose:

Retrieve all students

Example:

GET https://localhost:44384/api/Students

Response:

```
[
  {
    "id":1,
    "name":"Subash",
    "age":47
```

```
},  
{  
  "id":2,  
  "name":"Ram",  
  "age":20  
}  
]
```

Used for:

- Displaying all students
- Loading student lists
- Searching data

2. POST /api/Students

Purpose:

Create a new student

Example request:

```
{  
  "name":"Hari",  
  "age":25,  
  "email":"hari@gmail.com",  
  "mobile":"9812345678"  
}
```

The API receives this data and saves it.

Flow:

Client

|

| POST Student Data

↓

StudentController

|

↓

Database

Used for:

- Registration forms
- Adding records

3. GET /api/Students/{id}

Purpose:

Get a single student by ID

Example:

GET /api/Students/1

Response:

```
{  
  "id":1,
```

```
"name":"Subash",  
"age":47  
}
```

{id} is a route parameter.

Example:

/api/Students/5

means:

Find student whose ID = 5

Used for:

- View details page
- Edit page loading

4. PUT /api/Students/{id}

Purpose:

Update existing student data

Example:

PUT /api/Students/1

Request body:

```
{  
  "id":1,  
  "name":"Subash Updated",  
  "age":48,  
  "email":"new@gmail.com",  
  "mobile":"9812345678"  
}
```

Meaning:

Old Student ID 1



Replace with new information

Used for:

- Edit profile
- Update records

5. DELETE /api/Students/{id}

Purpose:

Delete a student

Example:

DELETE /api/Students/2

Meaning:

Find student ID = 2

Remove it

Used for:

- Delete buttons

- Removing records

Complete CRUD Mapping

HTTP Method	URL	Operation	Purpose
GET	/api/Students	Read All	Show all students
POST	/api/Students	Create	Add new student
GET	/api/Students/{id}	Read One	View one student
PUT	/api/Students/{id}	Update	Edit student
DELETE	/api/Students/{id}	Delete	Remove student

Why Swagger is useful?

Normally, to test an API you need:

- Postman
- Frontend application
- Manual HTTP requests

Swagger gives you a ready-made testing interface.

Example:

Without Swagger:

React App

|

ASP.NET Core API

|

Database

With Swagger:

Swagger UI

|

ASP.NET Core API

|

Database

In Real Applications

Example: Student Management System

Add Student Page

Frontend sends:

POST /api/Students

↓

API saves student

Student List Page

Frontend sends:

GET /api/Students

↓

API returns JSON

Edit Student Page

Frontend sends:

GET /api/Students/5

↓

Load student information

Save Changes

Frontend sends:

PUT /api/Students/5

↓

Update database

Delete Student

Frontend sends:

DELETE /api/Students/5

↓

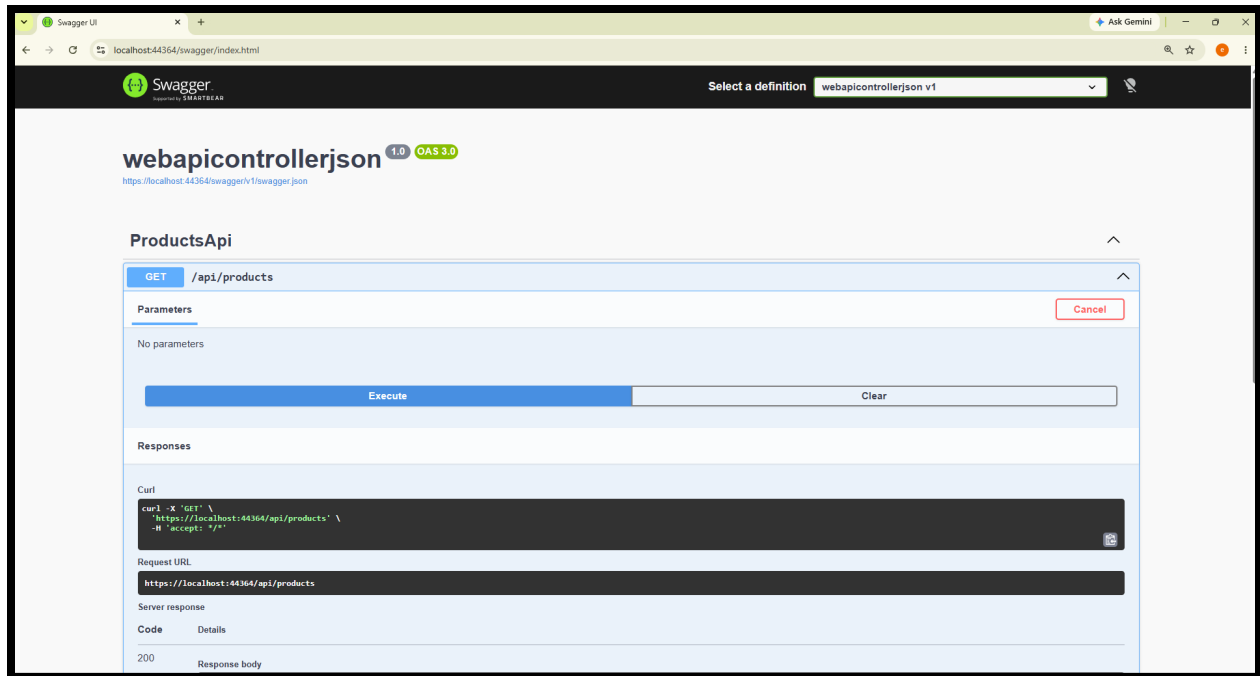
Remove record

Project: Online Shopping Product Management System

ASP.NET Core MVC Application + Web API Controller + JSON

In this version:

- ✓ ASP.NET Core MVC used for user interface
- ✓ API Controller handles data requests
- ✓ JSON used for communication
- ✓ Data stored using C# List collection



Dependency Injection and IOC containers

1. What is Dependency Injection (DI)?

Dependency Injection (DI) is a design pattern used to provide an object with the dependencies it needs instead of creating them inside the object itself.

In simple words:

A class does not create the objects it depends on; another system provides those objects.

Without Dependency Injection (Tightly Coupled)

Example:

```
public class OrderService
{
    private EmailService emailService;

    public OrderService()
    {
        emailService = new EmailService();
    }

    public void PlaceOrder()
    {
        emailService.SendEmail();
    }
}

public class EmailService
{
}
```

```

public void SendEmail()
{
    Console.WriteLine("Email Sent");
}
}

```

Problem:

- OrderService creates EmailService itself.
- It is tightly coupled.
- Difficult to replace or test.

Example:

- Today you use Email.
- Tomorrow you want SMS.
- You must modify OrderService.

2. Dependency Injection Approach

Instead of creating the dependency, pass it through the constructor.

Step 1: Create Interface

```

public interface IMessageService
{
    void SendMessage();
}

```

Step 2: Implement Interface

```

public class EmailService : IMessageService
{
    public void SendMessage()
    {
        Console.WriteLine("Email Sent");
    }
}

```

Step 3: Inject Dependency

```

public class OrderService
{
    private readonly IMessageService messageService;

    public OrderService(IMessageService messageService)
    {
        this.messageService = messageService;
    }

    public void PlaceOrder()
    {
        messageService.SendMessage();
    }
}

```

```
}  
}
```

Now OrderService does not know:

- How email is sent
- Which service is used

It only knows:

IMessageService

3. Types of Dependency Injection

1. Constructor Injection (Most Common)

Dependency is provided through constructor.

```
public class ProductController  
{  
    private readonly IProductService service;  
  
    public ProductController(IProductService service)  
    {  
        this.service = service;  
    }  
}
```

Used in ASP.NET Core applications.

2. Property Injection

Dependency is assigned through property.

```
public class ProductController  
{  
    public IProductService Service { get; set; }  
}
```

Less common.

3. Method Injection

Dependency is passed through method parameter.

```
public void Save(IProductService service)  
{  
    service.SaveProduct();  
}
```

4. What is IoC (Inversion of Control)?

IoC (Inversion of Control) is a principle where control of object creation is transferred from the programmer's code to a framework or container.

Normally:

Application Code

```
|  
|
```

Creates Objects



Dependencies

With IoC:

IoC Container



Creates Objects



Provides Dependencies

Example:

Without IoC:

Controller



creates



Service



creates



Repository

With IoC:

IoC Container



Controller



Service



Repository

5. What is an IoC Container?

An **IoC Container** is a framework component that:

- Creates objects
- Manages dependencies
- Injects required services automatically
- Controls object lifetime

ASP.NET Core has a built-in IoC container.

Example:

Interface



Implementation

|
IoC Container
|
Controller receives object

6. Dependency Registration in ASP.NET Core

In ASP.NET Core, dependencies are registered in:

Program.cs

Example:

```
builder.Services.AddTransient<IMessageService, EmailService>();
```

Now ASP.NET Core knows:

When someone asks for:

IMessageService

Give:

EmailService

7. Service Lifetime in IoC Container

ASP.NET Core provides three main lifetimes:

1. Transient

Creates a new object every time.

```
builder.Services.AddTransient<IEmailService, EmailService>();
```

Example:

Request 1 → New Object

Request 2 → New Object

Request 3 → New Object

Use for:

- Lightweight services
- Stateless operations

2. Scoped

Creates one object per HTTP request.

```
builder.Services.AddScoped<IOrderService, OrderService>();
```

Example:

Browser Request

|
|
One Service Instance

|

Response

Commonly used for:

- Database context
- Business services

3. Singleton

Creates only one object for the entire application.

`builder.Services.AddSingleton<ICacheService, CacheService>();`

Example:

Application Start

|

|

Single Object

|

All Users Share It

Used for:

- Configuration
- Caching
- Logging

8. Example: ASP.NET Core MVC with DI

Service Interface

```
public interface IStudentService
{
    List<string> GetStudents();
}
```

Service Class

```
public class StudentService : IStudentService
{
    public List<string> GetStudents()
    {
        return new List<string>()
        {
            "Ram",
            "Hari",
            "Sita"
        };
    }
}
```

Register Service

Program.cs

```
builder.Services.AddScoped<IStudentService, StudentService>();
```

Controller

```
public class StudentController : Controller
```

```
{
```

```
    private readonly IStudentService service;
```

```
    public StudentController(IStudentService service)
```

```
    {
```

```
        this.service = service;
```

```
    }
```

```
    public IActionResult Index()
```

```
    {
```

```
        var students = service.GetStudents();
```

```
        return View(students);
```

```
    }
```

```
}
```

Flow:

User Request

|

↓

StudentController

|

↓

IoC Container

|

↓

StudentService

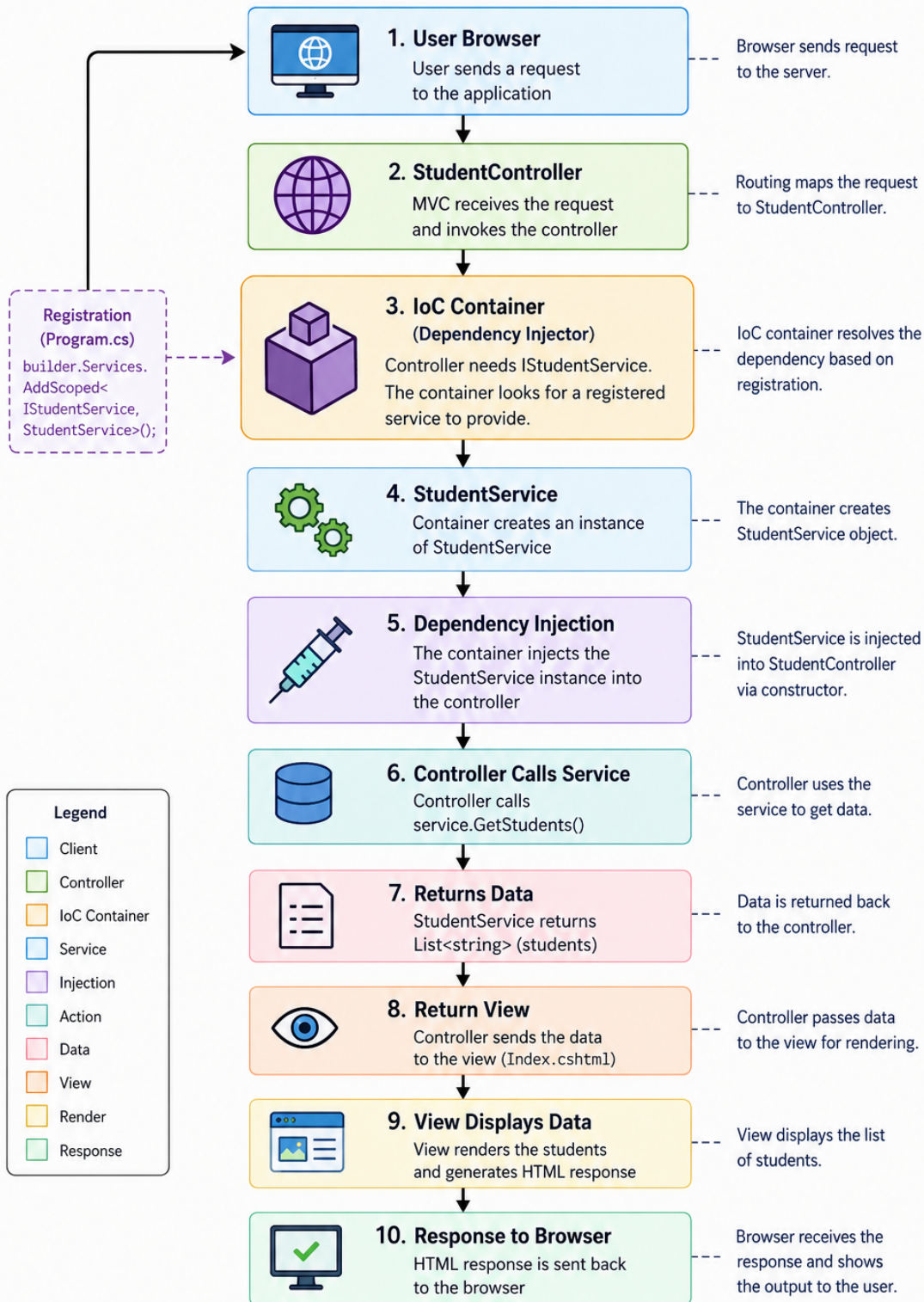
|

↓

Returns Data



How Dependency Injection Works



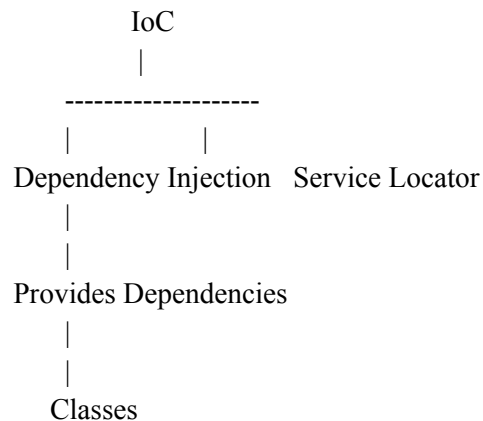
Summary

Dependency Injection allows the framework (IoC Container) to provide the required dependencies to a class automatically, promoting loose coupling and testability.

9. Advantages of Dependency Injection

Advantage	Explanation
Loose Coupling	Classes depend on interfaces, not implementations
Easy Testing	Dependencies can be replaced with mock objects
Maintainability	Code changes are easier
Reusability	Services can be reused
Better Architecture	Follows SOLID principles

10. DI + IoC Relationship



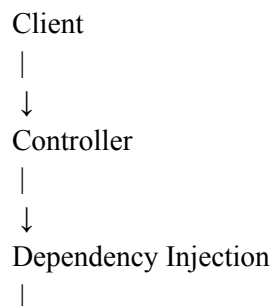
IoC is the principle.

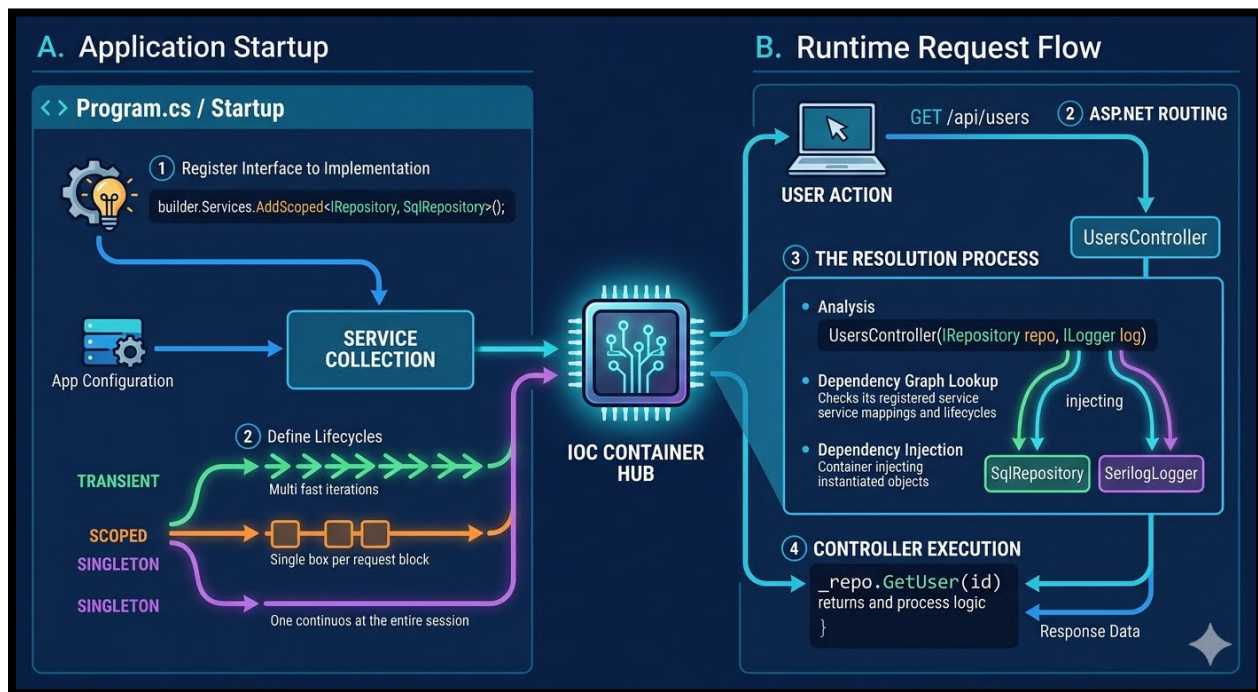
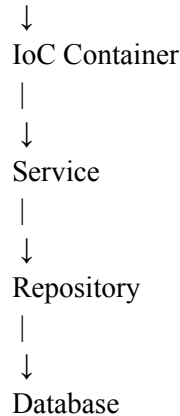
Dependency Injection is the technique used to achieve IoC.

Key Takeaways

- **DI** means providing dependencies from outside the class.
- **IoC** means transferring object creation control to a framework.
- **IoC Container** manages object creation and lifetime.
- ASP.NET Core has a built-in IoC container.
- Services are registered in Program.cs.
- Three lifetimes:
 - **Transient** → New object every time
 - **Scoped** → One object per request
 - **Singleton** → One object for entire application

ASP.NET Core Architecture:





Project 1: Hospital Appointment Management System

Scenario

A hospital's appointment system was built with AppointmentService directly creating its own SQLServerDatabase, SMSReminder, and BillingCalculator objects internally. Now the hospital wants to switch databases, add email reminders, and unit test billing logic — all without rewriting the core service.

Objectives

- Identify tight coupling issues in the existing design
- Refactor using Constructor/Setter Injection
- Configure and resolve dependencies through an IoC container
- Enable easy testing via mock dependencies

Tasks

1. Build the tightly-coupled baseline version and document its problems.

2. Define interfaces: `IDatabase`, `IReminderService`, `IBillingCalculator`.
3. Refactor `AppointmentService` to receive dependencies via constructor injection.
4. Register services in an IoC container (Spring, .NET Core DI, or similar) with appropriate lifetimes (Singleton/Scoped/Transient).
5. Add a new reminder type (Email) and swap it in via configuration — with zero changes to `AppointmentService`.
6. Write unit tests for appointment booking and billing logic using mock/fake dependencies.

Deliverables

- Source code (before/after refactor)
- Dependency diagram (tightly coupled vs loosely coupled)
- Report explaining DIP vs DI vs IoC
- Unit tests demonstrating improved testability

Project 2: E-Commerce Order Processing System

Scenario

An e-commerce startup's `OrderService` hardcodes a specific payment gateway (`CreditCardProcessor`), shipping provider (`FedExShipping`), and invoice generator. The business now wants to support multiple payment methods, multiple couriers, and A/B test invoice formats — with new providers added regularly and minimal code changes.

Objectives

- Apply Strategy Pattern with DI to select implementations at runtime
- Use IoC container configuration to bind interfaces to implementations
- Demonstrate Open/Closed Principle by adding new providers without touching existing code
- Understand container-managed object lifecycles

Tasks

1. Define abstractions: `IPaymentGateway`, `IShippingProvider`, `IInvoiceGenerator`.
2. Implement multiple concrete classes for each (e.g., `CreditCardProcessor`, `PayPalProcessor`; `FedExShipping`, `DHLShipping`).
3. Register all implementations in an IoC container, resolving the correct one based on user selection (e.g., using named/keyed registrations).
4. Inject dependencies into `OrderService` via its constructor rather than instantiating them directly.
5. Add a new payment provider (e.g., `StripeProcessor`) live, proving no existing class needs modification.
6. (Optional/advanced) Add a logging decorator around `IPaymentGateway` to log every transaction without modifying existing gateway classes.

Deliverables

- Working order processing simulator (console or simple UI)
- Dependency/composition diagram
- Comparison table: manual DI vs container-managed DI
- Reflection report on testability and extensibility gains

Controllers and Actions in ASP.NET Core MVC

What is a Controller?

A **Controller** is a C# class responsible for handling incoming HTTP requests from users. It receives the request, performs any required processing (often by calling models or services), and returns an appropriate response such as a View, JSON data, a file, or a redirect.

Controllers act as the **middle layer** between the user interface (View) and the application's business logic (Model/Services).

Every controller in ASP.NET Core MVC:

- Is a C# class.
- Inherits from the Controller base class (or ControllerBase for Web APIs).
- Is typically stored in the **Controllers** folder.
- Has a class name ending with **Controller** (e.g., StudentController).

What is an Action?

An **Action** is a public method inside a controller that responds to a specific URL request. When a request reaches a controller, ASP.NET Core MVC executes the appropriate action method.

Each action performs a specific task, such as:

- Displaying a page
- Saving data
- Updating records
- Deleting records
- Returning JSON
- Redirecting to another page

An action usually returns an object implementing IActionResult.

Relationship Between Controller and Action

Browser

|

| Request

v

Controller

|

| Calls

v

Action Method

|

| Performs Logic

v

Returns Response

(View, JSON, Redirect, File)

How MVC Uses Controllers and Actions

Suppose the browser requests:

<https://localhost:5001/Student/Index>

MVC Routing interprets this as:

Controller : StudentController

Action : Index()

ASP.NET Core creates the controller object and executes the Index() action.

Creating a Controller in Visual Studio

Method 1: Using Visual Studio

1. Right-click the **Controllers** folder.
2. Select **Add → Controller**.
3. Choose **MVC Controller - Empty**.
4. Name it **StudentController**.
5. Click **Add**.

Visual Studio creates:

```
using Microsoft.AspNetCore.Mvc;
```

```
namespace MyProject.Controllers
{
    public class StudentController : Controller
    {

    }
}
```

Method 2: Create Manually

Inside the **Controllers** folder create

StudentController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
namespace MyProject.Controllers
{
    public class StudentController : Controller
    {

    }
}
```

Creating Action Methods

Inside the controller add public methods.

Example:

```
public IActionResult Index()
{
    return View();
}
```

Another action

```
public IActionResult About()
{
```

```

        return View();
    }
    Another action
    public IActionResult Contact()
    {
        return View();
    }

```

Each action responds to a different URL.

Complete Example

using Microsoft.AspNetCore.Mvc;

```

namespace MyProject.Controllers
{
    public class StudentController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        public IActionResult About()
        {
            return View();
        }

        public IActionResult Contact()
        {
            return View();
        }
    }
}

```

URL Routing

URL	Controller	Action
/Student/Index	StudentController	Index()
/Student/About	StudentController	About()
/Student/Contact	StudentController	Contact()

Action Returning Data

An action can send data to a View.

```
public IActionResult Index()
{
    ViewBag.Message = "Welcome to ASP.NET Core MVC";

    return View();
}
View
<h2>@ViewBag.Message</h2>
Output
Welcome to ASP.NET Core MVC
```

Action Returning JSON

```
public IActionResult Student()
{
    return Json(new
    {
        Id = 1,
        Name = "Ram",
        Age = 20
    });
}
```

```
Output
{
  "id":1,
  "name":"Ram",
  "age":20
}
```

Action Returning a File

```
public IActionResult Download()
{
    return File("sample.pdf",
        "application/pdf");
}
```

Action Redirecting

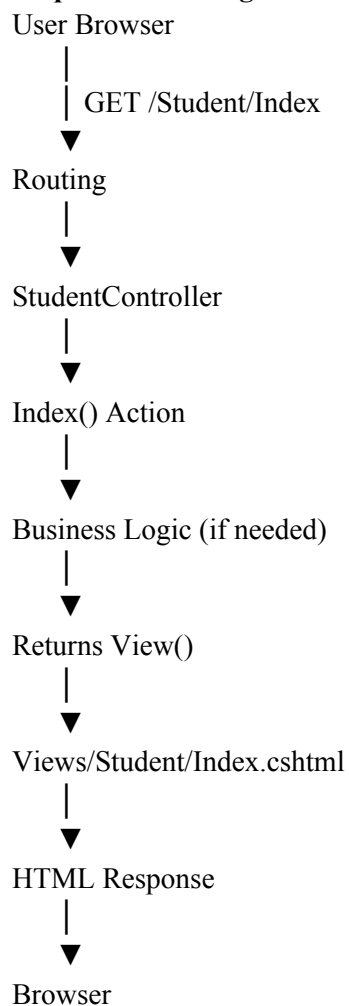
```
public IActionResult Home()
{
    return RedirectToAction("Index");
}
```

Common Return Types

Method	Purpose
View()	Returns a View

View(model)	Returns a View with data
Json()	Returns JSON data
Content()	Returns plain text
File()	Returns a downloadable file
Redirect()	Redirects to a URL
RedirectToAction())	Redirects to another action
NotFound()	Returns HTTP 404
BadRequest()	Returns HTTP 400
Ok()	Returns HTTP 200 (commonly used in APIs)

Request Processing Flow



Action Result Types in ASP.NET Core MVC

What is an Action Result?

An **Action Result** is the value returned by an action method in an ASP.NET Core MVC controller. It tells the MVC framework **what response should be sent back to the client (browser or application)** after processing a request.

Most MVC action methods return the IActionResult interface because it provides flexibility to return different types of responses.

Example:

```
public IActionResult Index()
{
    return View();
}
```

Here, the action returns a **View Result**, which renders an HTML page.

Why Do We Use Action Results?

Action Results allow a controller to return different kinds of responses depending on the request.

Common responses include:

- Displaying a View (HTML page)
- Returning JSON data
- Downloading a file
- Redirecting to another page
- Returning plain text
- Returning HTTP status codes (200, 404, 400, etc.)

Types of Action Results

1. ViewResult

Purpose:

Returns an HTML view to the browser.

Example:

```
public IActionResult Index()
{
    return View();
}
```

Output:

Views/Student/Index.cshtml

Use when:

- Displaying web pages
- Showing forms
- Displaying reports

2. ViewResult with Model

Passes data to the View.

```
public IActionResult Index()
{
    List<string> students =
    [
```

```

        "Ram",
        "Hari",
        "Sita"
    ];

    return View(students);
}
View:
@model List<string>

@foreach(var student in Model)
{
    <p>@student</p>
}
Output:
Ram
Hari
Sita

```

3. JsonResult

Returns JSON data.

```

public IActionResult Student()
{
    return Json(new
    {
        Id = 1,
        Name = "Ram",
        Age = 20
    });
}

```

Output:

```

{
  "id": 1,
  "name": "Ram",
  "age": 20
}

```

Use when:

- AJAX requests
- JavaScript applications
- APIs

4. ContentResult

Returns plain text or HTML.

```

public IActionResult Message()

```

```
{  
    return Content("Welcome to ASP.NET Core MVC");  
}
```

Output:

Welcome to ASP.NET Core MVC

You can also specify the content type:

```
return Content("<h2>Hello</h2>", "text/html");
```

5. FileResult

Returns a file for download or display.

```
public IActionResult Download()  
{
```

```
    return File(  
        "wwwroot/files/sample.pdf",  
        "application/pdf",  
        "sample.pdf");  
}
```

Output:

Downloads sample.pdf

Supported file types include:

- PDF
- Images
- Word documents
- Excel files
- ZIP files

6. RedirectResult

Redirects to another URL.

```
public IActionResult Google()  
{
```

```
    return Redirect("https://www.google.com");  
}
```

Browser redirects to:

<https://www.google.com>

7. RedirectToActionResult

Redirects to another action within the application.

```
public IActionResult Home()  
{
```

```
    return RedirectToAction("Index");  
}
```

Redirect to another controller:

```
return RedirectToAction("Index", "Home");
```

8. RedirectToRouteResult

Redirects using a route.

```
return RedirectToRoute(new
{
    controller = "Home",
    action = "Index"
});
```

9. EmptyResult

Returns an empty response.

```
public IActionResult Test()
{
    return new EmptyResult();
}
```

Output:

No Content

10. NotFoundResult

Returns HTTP 404.

```
public IActionResult Details(int id)
{
    return NotFound();
}
```

Browser:

404 Not Found

11. BadRequestResult

Returns HTTP 400.

```
public IActionResult Save()
{
    return BadRequest();
}
```

Output:

400 Bad Request

12. OkResult

Returns HTTP 200.

```
public IActionResult Success()
{
    return Ok();
}
```

Output:

200 OK

Return data with OK:

```
return Ok(new
{
    Message = "Saved Successfully"
});
```

13. UnauthorizedResult

Returns HTTP 401.

```
public IActionResult Login()
{
    return Unauthorized();
}
```

Output:

401 Unauthorized

14. StatusCodeResult

Returns a custom HTTP status code.

```
public IActionResult Error()
{
    return StatusCode(500);
}
```

Output:

500 Internal Server Error

Flow of Action Results

Browser

| Request



Controller



Action Method



Select Action Result



— View()

— Json()

— File()

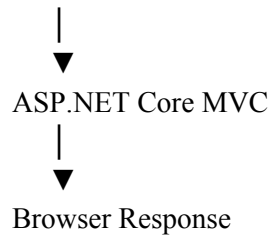
— Redirect()

— Content()

— NotFound()

— Ok()

— BadRequest()



Comparison of Common Action Result Types

Action Result	Returns	Typical Use
View()	HTML page	Display a web page
View(model)	HTML page with data	Pass model data to a view
Json()	JSON	AJAX calls and APIs
Content()	Plain text or HTML	Simple text or HTML responses
File()	File	Download or display files
Redirect()	URL redirect	Redirect to an external URL
RedirectToAction() ()	Redirect	Navigate to another action
RedirectToRoute()	Redirect	Redirect using route values
EmptyResult	Empty response	No content required
Ok()	HTTP 200	Successful request (often in APIs)
BadRequest()	HTTP 400	Invalid client request
Unauthorized()	HTTP 401	Authentication required
NotFound()	HTTP 404	Resource not found
StatusCode()	Custom HTTP status	Return a specific status code

When to Use Each Action Result

Scenario	Recommended Action Result
Show a web page	View()
Show a page with model data	View(model)
Return JSON to JavaScript	Json()
Download a PDF or image	File()
Return plain text	Content()
Redirect to another page	RedirectToAction()
Redirect to an external website	Redirect()
Resource does not exist	NotFound()
Invalid request	BadRequest()
Successful API response	Ok()
Authentication required	Unauthorized()
Return a custom HTTP status	StatusCode()

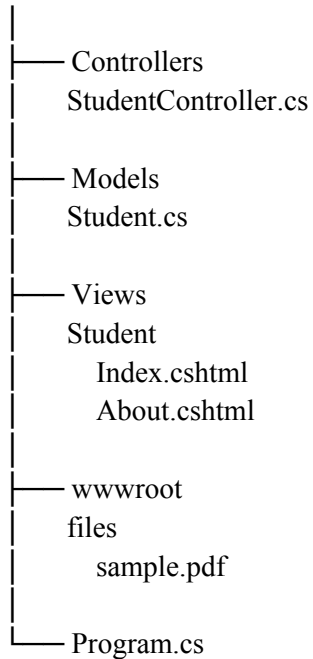
Assignment:

ASP.NET Core MVC project that demonstrates:

- Creating Actions
- Different Action Result Types

Project Structure

MvcActionResultDemo



Step 1: Create Model

Models/Student.cs

```
namespace MvcActionResultDemo.Models
{
    public class Student
    {
        public int Id { get; set; }

        public string Name { get; set; } = "";

        public int Age { get; set; }
    }
}
```

Step 2: Create Controller

Controllers/StudentController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using MvcActionResultDemo.Models;

namespace MvcActionResultDemo.Controllers
{
    public class StudentController : Controller
    {
        // Action 1 - View Result
        public IActionResult Index()
        {
            return View();
        }

        // Action 2 - View with Model
        public IActionResult StudentList()
        {
            List<Student> students =
            [
                new Student { Id = 1, Name = "Ram", Age = 20 },
                new Student { Id = 2, Name = "Hari", Age = 22 },
                new Student { Id = 3, Name = "Sita", Age = 21 }
            ];

            return View(students);
        }

        // Action 3 - JSON Result
        public IActionResult GetStudent()
        {
            Student student = new Student
            {
                Id = 1,
                Name = "Ram",
                Age = 20
            };

            return Json(student);
        }

        // Action 4 - Content Result
        public IActionResult Welcome()
        {
            return Content("Welcome to ASP.NET Core MVC!");
        }
    }
}
```

```
// Action 5 - Redirect Result
public IActionResult GoHome()
{
    return RedirectToAction("Index");
}

// Action 6 - NotFound Result
public IActionResult FindStudent(int id)
{
    if (id != 1)
        return NotFound();

    return Content("Student Found");
}

// Action 7 - BadRequest Result
public IActionResult SaveStudent(string name)
{
    if (string.IsNullOrEmpty(name))
        return BadRequest("Student name is required.");

    return Ok("Student Saved Successfully");
}

// Action 8 - File Result
public IActionResult DownloadFile()
{
    string path = Path.Combine(
        Directory.GetCurrentDirectory(),
        "wwwroot/files/sample.pdf");

    return PhysicalFile(
        path,
        "application/pdf",
        "sample.pdf");
}

// Action 9 - Empty Result
public IActionResult Empty()
{
    return new EmptyResult();
}

// Action 10 - Status Code Result
```

```

    public IActionResult ServerError()
    {
        return StatusCode(500, "Internal Server Error");
    }
}

```

Step 3: Create Index View

Views/Student/Index.cshtml

<h2>ASP.NET Core MVC Action Result Demo</h2>

```

<ul>
    <li><a href="/Student/StudentList">View Result</a></li>

    <li><a href="/Student/GetStudent">Json Result</a></li>

    <li><a href="/Student/Welcome">Content Result</a></li>

    <li><a href="/Student/GoHome">Redirect Result</a></li>

    <li><a href="/Student/FindStudent/1">NotFound Demo</a></li>

    <li><a href="/Student/SaveStudent?name=Ram">BadRequest / OK Demo</a></li>

    <li><a href="/Student/DownloadFile">File Result</a></li>

    <li><a href="/Student/Empty">Empty Result</a></li>

    <li><a href="/Student/ServerError">Status Code Result</a></li>
</ul>

```

Step 4: Create StudentList View

Views/Student/StudentList.cshtml

@model List<MvcActionResultDemo.Models.Student>

<h2>Student List</h2>

```

<table border="1" cellpadding="10">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Age</th>
    </tr>

```

```

@foreach(var student in Model)
{
    <tr>
        <td>@student.Id</td>
        <td>@student.Name</td>
        <td>@student.Age</td>
    </tr>
}

</table>

```

Step 5: Program.cs

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Student}/{action=Index}/{id?}");

app.Run();

```

Test URLs

URL	Action Result	Expected Output
/Student/Index	View()	Home page with links
/Student/StudentList	View(model)	Displays student table
/Student/GetStudent	Json()	JSON response
/Student/Welcome	Content()	Plain text message
/Student/GoHome	RedirectToAction())	Redirects to Index
/Student/FindStudent/1	Content()	"Student Found"
/Student/FindStudent/5	NotFound()	HTTP 404
/Student/SaveStudent?name=Ram	Ok()	"Student Saved Successfully"

/Student/SaveStudent	BadRequest()	HTTP 400 with error message
/Student/DownloadFile	PhysicalFile()	Downloads sample.pdf
/Student/Empty	EmptyResult	Empty response
/Student/ServerError	StatusCode()	HTTP 500

Project 1: Student Enrollment Portal (MVC Controllers & Actions)

Scenario

A university wants a web portal where students can view available courses, enroll, and check enrollment status. The system must handle both browser (HTML views) and API-style requests (JSON), and respond appropriately depending on success, failure, validation errors, or unauthorized access.

Objectives

- Design controllers following the MVC pattern
- Implement Create actions (GET for form display, POST for form submission)
- Use multiple Action Result types appropriately based on scenario
- Handle validation, redirection, and error responses correctly

Tasks

1. Create a CourseController with:
 - Index() → returns ViewResult listing all courses
 - Details(int id) → returns ViewResult or NotFoundResult if course doesn't exist
2. Create an EnrollmentController with:
 - Create() [GET] → returns ViewResult showing the enrollment form
 - Create(EnrollmentModel model) [POST] → validates input:
 - If invalid → return ViewResult with validation errors (re-render form)
 - If valid and successful → return RedirectToActionResult to a confirmation page
 - If course is full → return ViewResult with a custom error message, or a BadRequestResult if called via API
3. Add an API endpoint GetEnrollmentStatus(int studentId) that returns:
 - JsonResult with enrollment details if found
 - NotFoundResult (404) if student has no enrollment
 - UnauthorizedResult (401) if request lacks proper authorization token
4. Add a DownloadReceipt(int enrollmentId) action that returns a FileResult (PDF or text file) of the enrollment receipt.
5. Add a Cancel(int id) action that:
 - Returns RedirectToActionResult on success
 - Returns ForbidResult if the student tries to cancel someone else's enrollment
6. Implement a custom ActionResult (e.g., CsvResult) that exports the list of enrolled students as a downloadable CSV file.

Deliverables

- Full working MVC/Web API project (ASP.NET Core, Spring MVC, or similar)
- Table mapping each action to its possible Action Result types and the scenario that triggers each
- Report explaining the difference between ActionResult, IActionResult, and specific result types (ViewResult, JsonResult, RedirectToActionResult, etc.)
- Postman/API test collection demonstrating each result type being returned

Project 2: Online Job Application System (Create Actions & Result Handling)

Scenario

A recruitment platform allows companies to post jobs and candidates to apply. The system must support form-based submissions (with file upload for resumes), API responses for a mobile app, and different result types depending on validation, duplication checks, and authorization.

Objectives

- Implement full Create action lifecycle (GET/POST pattern)
- Return correct Action Result types based on business scenarios (success, duplicate, invalid, unauthorized, not found)
- Handle file uploads within a Create action
- Differentiate between View-based and API-based responses from the same controller logic

Tasks

1. Create a JobController:
 - Create() [GET] → ViewResult showing job posting form (for company/admin role only)
 - Create(JobModel model) [POST]:
 - If model state invalid → return ViewResult with errors
 - If job title already exists for that company → return ConflictResult (409) or a ViewResult with a duplicate warning
 - If successful → RedirectToActionResult to job listing page
 - Index() → ViewResult listing all jobs
2. Create an ApplicationController:
 - Apply(int jobId) [GET] → ViewResult with application form
 - Apply(int jobId, ApplicationModel model, IFormFile resume) [POST]:
 - Validate file type/size → if invalid, return BadRequestResult or ViewResult with error
 - If job no longer accepting applications → return ViewResult showing "Applications Closed" or StatusCodeResult(410) (Gone) for API clients
 - If successful → RedirectToActionResult to a "Thank You" page, and save uploaded resume to disk
3. Add GetApplicationStatus(int applicationId) as an API-style action:
 - Returns OkObjectResult (200) with JSON data if found
 - Returns NotFoundResult (404) if application doesn't exist
 - Returns UnauthorizedResult (401) if requester isn't the applicant or job owner
4. Add DownloadResume(int applicationId) → returns a FileStreamResult or PhysicalFileResult for the uploaded resume file.
5. Add Withdraw(int applicationId):
 - Returns NoContentResult (204) on successful API-based withdrawal
 - Returns RedirectToActionResult for browser-based withdrawal
6. (Advanced) Implement content negotiation: the **same** Details(int id) action should return a ViewResult when requested from a browser, but a JsonResult/OkObjectResult when requested with an Accept: application/json header.

Deliverables

- Working web application with file upload support
- Diagram/table showing each action, HTTP verb, and all possible Action Result outcomes

- Report comparing IActionResult vs JsonResult vs RedirectToActionResult vs FileResult vs status-code-based results (NotFoundResult, BadRequestResult, UnauthorizedResult, etc.)
- Short reflection: "Why does ASP.NET Core use IActionResult as a common return type instead of separate return types for every action?"

Rendering HTML with Views in ASP.NET Core MVC

What is a View?

A **View** is a Razor (.cshtml) file that generates the HTML sent to the user's browser. Views are responsible for presenting data and creating the user interface (UI).

In the MVC (Model-View-Controller) architecture:

- **Model** → Holds application data.
- **View** → Displays data as HTML.
- **Controller** → Receives requests, processes them, and selects which view to render.

A View should focus only on presentation and contain minimal business logic.

What Does "Rendering HTML" Mean?

Rendering HTML means converting a Razor View (.cshtml) into standard HTML that the browser can display.

Example:

Controller

```
public IActionResult Index()
{
    return View();
}
```

View (**Index.cshtml**)

```
<h2>Welcome to ASP.NET Core MVC</h2>
```

Generated HTML sent to the browser:

```
<h2>Welcome to ASP.NET Core MVC</h2>
```

The browser only receives HTML—it does **not** receive the Razor (@) syntax or C# code.

MVC Rendering Process

User Browser



Request (/Student/Index)



Routing

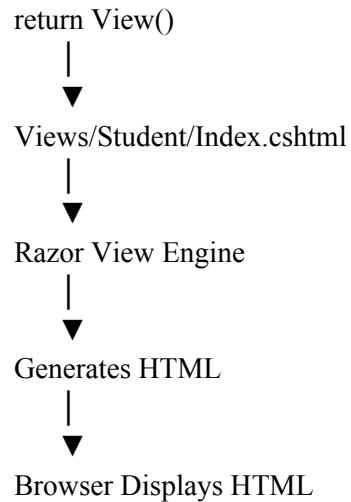


StudentController



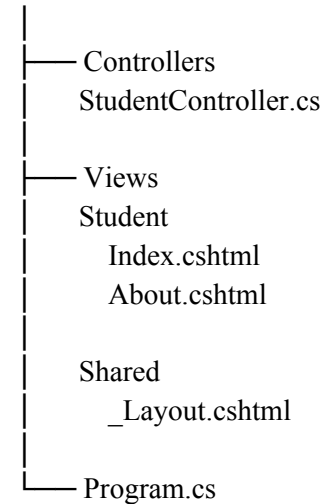
Index() Action





View Folder Structure

MyMvcProject



MVC automatically looks for:

Views

Student

Index.cshtml

when the controller executes:

return View();

inside

StudentController

Example 1: Rendering a Simple View Controller

using Microsoft.AspNetCore.Mvc;

```
namespace MyProject.Controllers
{
```

```

public class StudentController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}

```

View

Views/Student/Index.cshtml

```
<h1>Student Management System</h1>
```

```
<p>Welcome to ASP.NET Core MVC.</p>
```

Browser Output

Student Management System

Welcome to ASP.NET Core MVC.

Example 2: Passing Data Using ViewBag Controller

```

public IActionResult About()
{
    ViewBag.Title = "About Page";
    ViewBag.Message = "Welcome to ASP.NET Core MVC.";

    return View();
}

```

View

```
<h2>@ViewBag.Title</h2>
```

```
<p>@ViewBag.Message</p>
```

Browser Output

About Page

Welcome to ASP.NET Core MVC.

Example 3: Passing Data Using a Model Model

```

namespace MyProject.Models
{

```

```
public class Student
{
    public int Id { get; set; }

    public string Name { get; set; } = "";

    public int Age { get; set; }
}
}
```

Controller

```
using MyProject.Models;
```

```
public IActionResult Details()
{
    Student student = new Student
    {
        Id = 1,
        Name = "Ram",
        Age = 20
    };

    return View(student);
}
```

View

```
@model MyProject.Models.Student
```

```
<h2>Student Details</h2>
```

```
<p>ID : @Model.Id</p>
```

```
<p>Name : @Model.Name</p>
```

```
<p>Age : @Model.Age</p>
```

Browser Output

Student Details

ID : 1

Name : Ram

Age : 20

Example 4: Rendering Multiple Records

Controller

using MyProject.Models;

```
public IActionResult StudentList()
{
    List<Student> students =
    [
        new Student { Id = 1, Name = "Ram", Age = 20 },
        new Student { Id = 2, Name = "Hari", Age = 22 },
        new Student { Id = 3, Name = "Sita", Age = 21 }
    ];

    return View(students);
}
```

View

@model List<MyProject.Models.Student>

<h2>Student List</h2>

<table border="1" cellpadding="8">

```
<tr>
    <th>ID</th>
    <th>Name</th>
    <th>Age</th>
</tr>
```

@foreach(var student in Model)

```
{
    <tr>
        <td>@student.Id</td>
        <td>@student.Name</td>
        <td>@student.Age</td>
    </tr>
}
```

</table>

Browser Output

ID Name Age

1 Ram 20
2 Hari 22

Razor Syntax

Razor lets you combine C# code with HTML.

Display a variable:

```
@Model.Name
```

Use an if statement:

```
@if(Model.Age >= 18)
```

```
{
```

```
    <p>Adult</p>
```

```
}
```

Use a loop:

```
@foreach(var student in Model)
```

```
{
```

```
    <p>@student.Name</p>
```

```
}
```

View Rendering Flow

Browser



Controller Action



Creates Model



return View(model)



Razor View Engine



Combines HTML + C#



Generates HTML



Browser

Types of Views

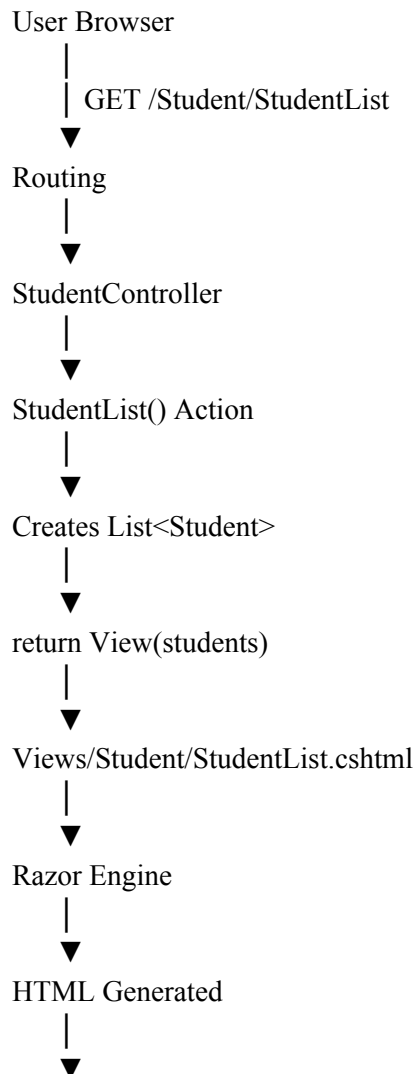
View Type	Description
-----------	-------------

Regular View	Displays a complete page
Partial View	Reusable section of a page
Layout View	Shared page template (header, footer, navigation)
View with Model	Displays strongly typed data
Empty View	Static page without model data

Advantages of Using Views

- Separates presentation from business logic.
- Makes applications easier to maintain.
- Supports reusable layouts and partial views.
- Uses Razor syntax to combine HTML and C#.
- Produces standard HTML that works in any web browser.

Complete Request Flow



Browser Displays Student List

Key Points

- A **View** is a .cshtml file used to generate HTML.
- The **Controller** chooses which view to render.
- The **Razor View Engine** processes Razor syntax (@) and produces HTML.
- Views can receive data using **Model**, **ViewBag**, or **ViewData**.
- The browser receives only the final HTML, not the Razor code or C# logic.
- By convention, return View(); in an action looks for a view with the same name as the action in the corresponding controller's folder under **Views**.

Rendering HTML with Views: Razor Syntax

Rendering HTML with Views: Razor Syntax (ASP.NET Core MVC .NET 8)

What is Razor Syntax?

Razor is the view engine used in **ASP.NET Core MVC**. It allows you to write **HTML and C# code together** in the same .cshtml file.

Razor uses the **@ (at sign)** to switch from HTML to C# code.

For example:

```
<h2>Welcome</h2>
```

```
<p>@DateTime.Now</p>
```

When the page is requested, the Razor engine executes the C# code and generates pure HTML that is sent to the browser.

Why Use Razor?

Razor makes it easy to:

- Display dynamic data.
- Write loops and conditional statements.
- Bind data from models.
- Create reusable layouts and partial views.
- Generate HTML dynamically.

Razor View Rendering Process

Browser Request



Controller



View (.cshtml)



Razor Engine



Executes C# Code



Generates HTML



Browser

Razor Syntax Basics

1. Display a Variable

Controller

```
public IActionResult Index()
{
    ViewBag.Name = "Ram";
```

```
    return View();
}
```

View

```
<h2>Hello @ViewBag.Name</h2>
```

Output

Hello Ram

2. Display Current Date and Time

```
<p>Today is @DateTime.Now</p>
```

Example Output

Today is 7/19/2026 6:00 PM

3. C# Code Block

Use @{ } for multiple C# statements.

```
@{
    string name = "Hari";
    int age = 22;
}
```

```
<p>Name: @name</p>
```

```
<p>Age: @age</p>
```

Output

Name: Hari

Age: 22

4. If Statement

```
@{
    int marks = 75;
}
```

```
@if (marks >= 40)
```

```
{
```

```
    <p>Pass</p>
```

```
}
```

```
else
```

```
{
```

```
    <p>Fail</p>
```

```
}
```

```
Output
```

```
Pass
```

```
|
```

5. Switch Statement

```
@{
```

```
    int day = 2;
```

```
}
```

```
@switch (day)
```

```
{
```

```
    case 1:
```

```
        <p>Sunday</p>
```

```
        break;
```

```
    case 2:
```

```
        <p>Monday</p>
```

```
        break;
```

```
    default:
```

```
        <p>Unknown Day</p>
```

```
        break;
```

```
}
```

```
Output
```

```
Monday
```

```
|
```

6. For Loop

```
@for(int i = 1; i <= 5; i++)
```

```
{
```

```
    <p>Number: @i</p>
```

```
}
```

```
Output
```

```
Number: 1
```

```
Number: 2
```

```
Number: 3
```

```
Number: 4
```

Number: 5

7. Foreach Loop

```
@{
    List<string> students =
    [
        "Ram",
        "Hari",
        "Sita"
    ];
}
```

```
@foreach(var student in students)
{
    <p>@student</p>
}
```

Output

Ram

Hari

Sita

8. Display Model Data

Model

```
public class Student
```

```
{
    public int Id { get; set; }

    public string Name { get; set; } = "";

    public int Age { get; set; }
}
```

Controller

```
public IActionResult Details()
```

```
{
    Student student = new Student
    {
        Id = 1,
        Name = "Ram",
        Age = 20
    };
}
```

```
    return View(student);
```

```
}
```

View

@model Student

<h2>Student Details</h2>

<p>ID : @Model.Id</p>

<p>Name : @Model.Name</p>

<p>Age : @Model.Age</p>

Output

Student Details

ID : 1

Name : Ram

Age : 20

9. Display List Using Model

Controller

```
public IActionResult StudentList()
```

```
{
```

```
    List<Student> students =
```

```
    [
```

```
        new Student { Id = 1, Name = "Ram", Age = 20 },
```

```
        new Student { Id = 2, Name = "Hari", Age = 22 },
```

```
        new Student { Id = 3, Name = "Sita", Age = 21 }
    ];
```

```
    return View(students);
```

```
}
```

View

@model List<Student>

<table border="1">

<tr>

<th>ID</th>

<th>Name</th>

<th>Age</th>

</tr>

@foreach(var student in Model)

```
{
```

```
    <tr>
```

```

        <td>@student.Id</td>
        <td>@student.Name</td>
        <td>@student.Age</td>
    </tr>
}
</table>

```

10. HTML + Razor Together

```
<h2>Student Information</h2>
```

```
<p>Name: @Model.Name</p>
```

```
<p>Age: @Model.Age</p>
```

```

@if(Model.Age >= 18)
{
    <strong>Adult Student</strong>
}

```

Output

Student Information

Name: Ram

Age: 20

Adult Student

11. Comments in Razor

HTML Comment (visible in page source)

```
<!-- HTML Comment -->
```

Razor Comment (removed before HTML is generated)

```
@* This is a Razor comment *@
```

12. Escape the @ Symbol

To display a literal @:

```
@@gmail.com
```

Output

@gmail.com

Common Razor Symbols

Symbol	Purpose	Example
@	Switch from HTML to C#	@Model.Name
@{ }	C# code block	@{ int x = 10; }
@if	Conditional logic	@if(age > 18)

@for	For loop	@for(int i=0; i<5; i++)
@foreach	Iterate through a collection	@foreach(var s in Model)
@switch	Switch statement	@switch(day)
@model	Declare the model type	@model Student
@Model	Access the model instance	@Model.Name
@ViewBag	Access dynamic data	@ViewBag.Title
@ViewData	Access dictionary-based data	@ViewData["Title"]
@* *@	Razor comment	@* Hidden comment *@

Complete Razor Rendering Flow

User Browser



Request /Student/Index



StudentController



Creates Model/ViewBag



return View(model)



Index.cshtml



Razor Engine



Executes C#

Replaces @Model

Processes Loops

Evaluates Conditions

Generates HTML



Browser Displays HTML

Advantages of Razor Syntax

- Simple and easy to read.
- Combines HTML and C# in one file.

- Strongly typed model support with @model.
- Built-in support for loops and conditional statements.
- Produces clean HTML for the browser.
- Helps keep presentation logic separate from application business logic.

Understanding Tag Helpers

Understanding Tag Helpers in ASP.NET Core MVC (.NET 8)

What are Tag Helpers?

Tag Helpers are server-side components in ASP.NET Core MVC that enable Razor views to generate HTML more easily by extending standard HTML elements with ASP.NET Core-specific attributes. They make Razor views cleaner, more readable, and easier to maintain than embedding complex C# code directly in HTML.

For example, instead of manually writing a URL:

```
<a href="/Student/Index">Students</a>
```

you can use a Tag Helper:

```
<a asp-controller="Student" asp-action="Index">Students</a>
```

ASP.NET Core generates the correct HTML automatically.

Why Use Tag Helpers?

Tag Helpers provide several benefits:

- Generate URLs automatically based on routing.
- Reduce hard-coded links.
- Improve code readability.
- Work naturally with HTML.
- Support IntelliSense in Visual Studio.
- Help prevent broken links when routes change.

How Tag Helpers Work

Browser Request



Controller



Razor View (.cshtml)



Tag Helpers



Generate HTML



Browser

Enabling Tag Helpers

In ASP.NET Core MVC projects, the **Views/_ViewImports.cshtml** file usually contains:

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

This imports the built-in ASP.NET Core MVC Tag Helpers into all Razor views.

Common Tag Helpers

1. Anchor Tag Helper (<a>)

Creates links based on MVC routing.

Razor

```
<a asp-controller="Student"
    asp-action="Index">
    Student List
</a>
```

Generated HTML

```
<a href="/Student/Index">
    Student List
</a>
```

2. Form Tag Helper

Generates the correct form action URL.

```
<form asp-controller="Student"
      asp-action="Save"
      method="post">

    <input type="submit" value="Save" />

</form>
```

Generated HTML

```
<form action="/Student/Save"
      method="post">

    <input type="submit" value="Save">

</form>
```

3. Input Tag Helper

Model

```
public class Student
{
    public string Name { get; set; } = "";

    public int Age { get; set; }
}
```

View

@model Student

<input asp-for="Name" />

<input asp-for="Age" />

Generated HTML

```
<input id="Name"
      name="Name"
      type="text">
```

```
<input id="Age"
      name="Age"
      type="number">
```

4. Label Tag Helper

<label asp-for="Name"></label>

Generated HTML

<label for="Name">Name</label>

5. Validation Message Tag Helper

Displays validation errors for the **Name** property.

6. Validation Summary Tag Helper

<div asp-validation-summary="All"></div>

Displays all validation errors.

Options:

- All
- ModelOnly
- None

7. Select Tag Helper

Model

public class Student

{

public string Course { get; set; } = "";

}

Controller

ViewBag.Courses = new List<string>

{

"BCA",

"BBS",

"BIT"

};

View

```
<select asp-for="Course"
        asp-items="new SelectList(ViewBag.Courses)">
</select>
```

Generated HTML

```
<select id="Course" name="Course">
```

```
<option>BCA</option>
```

```
<option>BBS</option>
```

```
<option>BIT</option>
```

```
</select>
```

8. Environment Tag Helper

Displays content only in a specific environment.

```
<environment include="Development">
```

```
<p>Development Environment</p>
```

```
</environment>
```

9. Partial Tag Helper

Renders a partial view.

```
<partial name="_StudentDetails" />
```

Equivalent to:

```
@await Html.PartialAsync("_StudentDetails")
```

Complete Example

Student Model

```
namespace MyProject.Models
```

```
{
```

```
    public class Student
```

```
    {
```

```
        public string Name { get; set; } = "";
```

```
        public int Age { get; set; }
```

```
    }
```

```
}
```

Controller

```
using Microsoft.AspNetCore.Mvc;
```

```
using MyProject.Models;
```

```

public class StudentController : Controller
{
    public IActionResult Create()
    {
        return View();
    }

    [HttpPost]
    public IActionResult Save(Student student)
    {
        return Content("Student Saved Successfully");
    }
}

```

View

@model Student

<h2>Add Student</h2>

<form asp-action="Save"
 asp-controller="Student"
 method="post">

<label asp-for="Name"></label>

<input asp-for="Name" />

<label asp-for="Age"></label>

<input asp-for="Age" />

<button type="submit">
 Save
</button>

</form>

Browser Output

Add Student

Name:

Age :

[Save]

Tag Helper Processing Flow

Browser Request



StudentController



return View()



Create.cshtml



Tag Helpers Process HTML



Generate Standard HTML



Browser Displays Form

Common Tag Helper Attributes

Tag Helper	Attribute	Purpose
Anchor	asp-controller	Specifies the target controller
Anchor	asp-action	Specifies the target action
Anchor	asp-route-id	Passes route values
Form	asp-action	Specifies the action that receives the form
Form	asp-controller	Specifies the controller
Input	asp-for	Binds an input to a model property
Label	asp-for	Creates a label for a model property
Validation	asp-validation-for	Displays validation messages
Validation	asp-validation-	Displays validation summary

Summary	summary	
Select	asp-for	Binds the selected value
Select	asp-items	Supplies the list of options
Partial	name	Specifies the partial view to render

Advantages of Tag Helpers

- Easier to read than mixing HTML and C# helper methods.
- Generate URLs based on MVC routing, avoiding hard-coded paths.
- Strongly typed support with asp-for.
- Reduce coding errors and improve maintainability.
- Work seamlessly with model binding and validation.
- Supported by IntelliSense in Visual Studio.

HTML Helpers vs. Tag Helpers

HTML Helpers	Tag Helpers
Use C# methods such as <code>@Html.TextBoxFor()</code>	Extend standard HTML elements with attributes
Less like regular HTML	Look and behave like normal HTML
Harder to read for complex pages	More readable and maintainable
Written as C# expressions	Written using HTML with asp-* attributes

Project: Simple Product Catalog Page

Scenario

A small shop wants a single web page that lists products, lets the admin add a new product via a form, and shows stock status clearly — built using Razor views and Tag Helpers only (no raw HTML forms/links).

Objectives

- Practice core Razor syntax (`@model`, `@foreach`, `@if`)
- Use built-in Tag Helpers for forms, inputs, links, and validation
- Build one simple custom Tag Helper

Tasks

1. Create a `ProductViewModel` (Name, Price, StockCount).
2. `Index.cshtml` — use `@model List<ProductViewModel>` and `@foreach` to list products in a table; use `@if` to show "In Stock" / "Out of Stock" text.
3. `Create.cshtml` — build the add-product form using:
 - `<form asp-controller="Product" asp-action="Create">`
 - `<input asp-for="Name" />`, `<label asp-for="Name" />`
 - `<span asp-validation-for="Price"></span>` for validation
4. Use the **Anchor Tag Helper** to link from Index to Create:
 `<a asp-controller="Product" asp-action="Create">Add Product</a>`
5. Build one **custom Tag Helper**, `<stock-badge count="5">`, that renders "In Stock" (green) or "Out of Stock" (red) automatically.

Deliverables

- Working Index and Create views
- One custom Tag Helper class + usage in a view

- Short note: raw HTML vs Tag Helper comparison (2–3 sentences)